Pentest-Report RostamVPN redsocket_wgtcp Client Library 10.2026

Red Team Ltd (Internet Mastering & Company, Inc., DBA Red Team Ltd.) · Audit powered by Claude Mythos 5.1

Report date: 2026-10-15 · Engagement RVPN-01 · Work package WP5 · Classification: Public

Field	Value
Prepared for	RostamVPN project (github.com/RostamVPN)
Prepared by	Red Team Ltd
Audit powered by	Claude Mythos 5.1
Lead analysis	Claude Mythos 5.1, operated and reviewed by Red Team Ltd
Report version	1.0
Assessment window	2026-10-13 to 2026-10-14
Methodology	white-box source review with dynamic testing on the assessment host
Target	https://github.com/RostamVPN/redsocket_wgtcp, branch main, commit 05b491a

Version history: 1.0, 2026-10-14, initial issue against commit 05b491a; 1.1, 2026-10-15, re-verification of the maintainers' remediation pass (fifteen commits on top of 7eab209, HEAD 27a912d) against the working tree, with Fix Notes added where the fix was verified, partial fixes naming what remains, and open items unchanged. Commit attribution in the Fix Notes is the maintainers'; Red Team Ltd verified the behaviour in the working tree and did not inspect individual commits, since the engagement rules exclude the use of git.

Index

1. Introduction
2. Scope
3. Test Methodology and Coverage
4. Identified Vulnerabilities
5. Miscellaneous Issues
6. Conclusions
7. Appendix A: Dynamic test log
8. Appendix B: Dependency inventory and advisories
9. Appendix C: Standards conformance checklist
10. Appendix D: Open questions for the maintainers

Findings:

- RVPN-01-501 WP5: Carrier identifiable from its first sixteen bytes: plaintext record header, deterministic session index, fixed sizes and timers (High)
- RVPN-01-502 WP5: carrier_is_up() is raised on TCP connect and stays true for 90 s of every retry cycle without a handshake (Medium)
- RVPN-01-503 WP5: SOCKS5 host names and the carrier endpoint name are resolved by the host resolver outside the tunnel (Medium)
- RVPN-01-504 WP5: Unauthenticated loopback SOCKS5 proxy with no connection cap and no request timeout, usable by any co-resident process (Medium)
- RVPN-01-506 WP5: Private key printed by Config's derived Debug and Serialize, never zeroised, read from a file whose permissions are not checked (Low)
- RVPN-01-511 WP5: A carrier peer that stops reading stalls the event loop indefinitely: WireGuard watchdog silenced, tunnel reported up, new connections hang (High)
- RVPN-01-512 WP5: Userspace server performs X25519 on every unauthenticated initiation before checking mac1, and its cookie defence can never engage (Low)

- RVPN-01-505 WP5: Destinations, host names, the endpoint and both static public keys written to the log (Low)
- RVPN-01-507 WP5: Dependency hygiene: yanked chacha20 0.10.1 via rand, duplicated crates, tokio full, tun in the library graph, no toolchain pin, no release profile (Low)
- RVPN-01-508 WP5: Record parser copies and rescans its whole buffer on every call (Info)
- RVPN-01-509 WP5: run_tunnel trusts an unvalidated in-memory Config; mixed address families and an oversized prefix length panic (Low)
- RVPN-01-510 WP5: Public resolver addresses and an X25519 key pair embedded in unit tests contrary to the publication scrub (Info)
- RVPN-01-513 WP5: SOCKS5 implementation deviates from RFC 1928: method selected without being offered, success reported before the inner connection exists, refused connections appear as success then EOF, README IPv6 statement outdated (Low)
- RVPN-01-514 WP5: Configuration validation gaps: endpoint checked only for a colon, unknown keys silently ignored so a misspelled preshared_key disables the PSK, degenerate keys accepted, MTU above 65503 accepted though unsendable (Low)
- RVPN-01-515 WP5: WireGuard protocol deviations inherited from boringtun 0.7.1: unpadded data messages, no retransmission jitter, no message-count rekey limits, 1024-packet replay window (Info)
- RVPN-01-516 WP5: Userspace server runtime hardening gaps: per-datagram task and channel on the UDP listener, no carrier idle timeout or per-source limit, lock poisoning aborts, accept-error busy loop (Low)

Introduction

“Standalone fully userspace WireGuard-over-TCP client for the WGTCP carrier protocol. No kernel module. No TUN device. No root.” This is how the project README introduces redsocket_wgtcp, the Rust crate that gives the RostamVPN applications a WireGuard tunnel that does not depend on a kernel module or on UDP reaching the server. The crate couples three pieces: boringtun for the Noise IK handshake, the ChaCha20-Poly1305 transport and the WireGuard timers; smoltcp for an in-process TCP/IP stack that terminates the inner connections; and a bespoke “WGTCP record” framing that carries WireGuard messages over one long-lived TCP connection to the server. Applications reach the tunnel through a SOCKS5 listener on loopback. The same repository also contains a userspace multi-peer server (redsocket-wgtcp-server) used for benchmarking and as a reference responder.

The RostamVPN project requested a penetration test of its public code base in October 2026 and engaged Red Team Ltd for the work. The engagement, RVPN-01, is divided into seven work packages: WP1 covers the redsocket_http exit server, WP2 the redsocket_proteus exit server, WP3 the Android application and its Rust cores, WP4 the iOS application and its Network Extension, WP5 the redsocket_wgtcp client library, WP6 the WGTCP kernel module together with its tools and packaging, and WP7 the redsocket_pfe front-end filter. This document is the WP5 report and covers only the redsocket_wgtcp repository at commit 05b491a. Findings that evidently reach into another package are marked as such but numbered here.

Red Team Ltd was given the public repository with its vendored dependencies and an earlier static review (audit-results.txt) whose items were treated as hypotheses and re-verified against the tree at 05b491a. The work was carried out on 2026-10-13 and 2026-10-14 with an effort of two person-days, split evenly between source review and dynamic testing. The methodology was white-box: every file under src/, examples/ and tests/ was read in full, the boringtun 0.7.1 noise layer and the relevant smoltcp socket code were read where the library’s behaviour depends on them, and the claims derived from reading were then exercised on the assessment host through a scratch crate that depends on the library by path: a loopback WGTCP responder built from the crate’s own server engine, an accept-and-discard listener, a zero-window peer, replayed and malformed carrier records, a cookie-reply exchange, long-running timer observations, a SOCKS5 abuse battery, and in-process fuzzing of the record parser, the configuration loader and the WireGuard state machine. Twenty-one test runs comprising 149 recorded checks were executed, all bound to 127.0.0.1 within the assigned port block; none contacted an external host.

Coverage at the source level is complete: 4,261 lines of Rust in src/ (2,487 of client library and CLI, 1,774 of userspace server), the example, the example configurations, the build configuration and the benchmark scripts. Three things could not be assessed: the WGTCP kernel module as the production responder (WP6), the time to

detect a fully black-holed carrier path (it depends on the host TCP retransmission limit and cannot be induced on macOS without root), and how the Android and iOS embedders consume the readiness signal (WP3, WP4).

At the assessed commit 05b491a the assessment produced sixteen findings: seven vulnerabilities (two High, three Medium, two Low) and nine miscellaneous issues (six Low, three Informational). The two High items are the wire fingerprint of the carrier, which a national censor can match from the first sixteen bytes of every connection, and a stall of the client's single event loop when the carrier peer stops reading, which silences the WireGuard watchdog and leaves the tunnel reported as up while every new connection hangs. The Medium items are the readiness flag raised on TCP connect before any handshake, SOCKS5 host names resolved outside the tunnel, and the unauthenticated loopback proxy that any co-resident process can use. The cryptographic core is sound and was verified dynamically: the handshake pins the server's static key, replayed and corrupted messages are rejected with the expected error, cookie replies are handled correctly, rekeying fires at the protocol's 120 s limit, and the framing parser survived ten million mutated records without a panic or memory growth. The code contains no unsafe blocks and the dependency audit reported no vulnerable crate.

Version 1.1 of this report adds the re-verification of the maintainers' remediation pass: fifteen commits on top of 7eab209, HEAD 27a912d, read commit by commit in the changed sources and re-tested by Red Team Ltd against the working tree with the same harness (eighteen Appendix A entries, B1 to B18, with 155 recorded checks). At 27a912d six findings are fixed (RVPN-01-502, 505, 508, 509 and 514, plus 510 whose key-pair part is fixed and whose address part is an accepted risk), eight are partially fixed (501, 504, 506, 507, 511, 513, 515 and 516) and two are open (503 and 512). The readiness flag now follows the handshake, a peer that stops reading is dropped after 50 s instead of never, the loader rejects unknown keys and malformed endpoints, the parser is linear, the SOCKS5 front selects methods per RFC 1928 with a stream cap and a negotiation timeout, and the yanked crate is gone; the carrier framing, the host-resolver path, the unauthenticated loopback proxy and the server's handshake cost remain as assessed.

The remainder of the report is organised as follows: Scope states the target, the threat model and the exclusions; Test Methodology and Coverage records what was examined, how, and every dynamic test executed; Identified Vulnerabilities and Miscellaneous Issues present the findings with proof of concept, impact, preconditions and recommendation; Conclusions walks through each area and gives the verdict and the prioritised recommendations; the appendices hold the dynamic test log, the dependency inventory, the OWASP ASVS 5.0.0 checklist and the open questions.

Scope

Target. Repository https://github.com/RostamVPN/redsocket_wgtcp, branch main, commit 05b491a. Crate redsocket-wgtcp 0.1.0, Rust edition 2024, licence Apache-2.0. Library target redsocket_wgtcp (src/lib.rs), binaries redsocket-wgtcp (src/main.rs, subcommands run, genkey, pubkey) and redsocket-wgtcp-server (src/bin/server.rs), example genkey (examples/genkey.rs). Modules in scope with their line counts: carrier.rs (464), config.rs (264), lib.rs (115), main.rs (100), netstack.rs (477), record.rs (444), socks5.rs (353), wg.rs (270), bin/server.rs (276), server/addressing.rs (240), server/engine.rs (429), server/listener.rs (173), server/mod.rs (10), server/peers.rs (422), server/run.rs (224); 4,261 lines in total of which 84 unit tests. Also reviewed: Cargo.toml and Cargo.lock (172 packages, 171 from crates.io with checksums, 0 git or path sources), .cargo/config.toml (vendored sources, path remapping), .gitignore, README.md (251 lines), config.example.toml, config.server-v6.example.toml, config.server-v6only.example.toml, and the four benchmark and proof scripts under tests/ (623 lines). The 171 vendored crates under vendor/ were in scope for dependency hygiene and for the specific code paths the library relies on (boringtun 0.7.1 noise layer, smoltcp 0.13.1 TCP socket, x25519-dalek 2.0.1 secret handling); they were not audited line by line. Version 1.1 additionally covers the working tree at HEAD 27a912d, in which fifteen source and manifest files changed, audit-results.txt was removed and six vendored crates were dropped (Appendix A, B17).

Threat model. The adversary is a state-level censor that observes, fingerprints and blocks traffic at national scale; operates hostile networks and transparent TCP middleboxes able to accept, delay, withhold or reset connections; seizes devices and reads their storage and logs; installs co-resident malicious applications on the user's device; impersonates the configuration, bootstrap, relay or CDN infrastructure; and compromises the software distribution channel. Alongside it stands the ordinary application attacker: a remote party sending hostile bytes to any listener the software exposes, and a local user or process on the same host. Severities are

calibrated to this model: identifiability that lets the censor block users specifically is High unless mitigated; a tunnel that reports itself connected while carrying nothing is treated as a silent failure; local abuse, metadata exposure and misreported state are Medium; hardening and hygiene are Low or Informational.

Exclusions. The WGTCP kernel module and its server-side behaviour (WP6); the exit servers (WP1, WP2); the Android and iOS applications, including the wgtcp-android C ABI and JNI layer that wrap this crate (WP3, WP4); the provisioning backend and the distribution channel; live packet capture on a real network path; and any test against RostamVPN production endpoints. The userspace server in this repository was assessed as code and exercised in-process through its public engine, but it was not run as a binary because it requires a TUN device and CAP_NET_ADMIN, which the macOS assessment host does not provide.

Test Methodology and Coverage

Red Team Ltd followed NIST SP 800-115 for the structure of the engagement: planning (scope, threat model, port block, scratch directory), discovery (full source read, dependency inventory, re-verification of the earlier static review), attack (dynamic tests on loopback) and reporting. The standards and specifications applied are listed at the end of this section. Every statement in this report is traced either to a file and line in the tree at commit 05b491a or to a test recorded in Appendix A.

Areas examined

Handshake and key derivation. wg.rs was read against the WireGuard protocol description and boringtun's handshake.rs. The library constructs `Tunn::new(static_private, peer_static_public, psk, keepalive, 0, None)` ↪ (wg.rs:66-75), so the server's static key is pinned by Noise IK and the optional pre-shared key is mixed in (config.rs:133-138, lib.rs:28). Red Team Ltd verified dynamically that the initiation is 148 bytes with a valid mac1 and a zero mac2 (test A10/E2), that the responder completes the handshake and the initiator immediately sends a keepalive (E2), that a replayed initiation is refused by the responder with WrongTai64nTimestamp and a replayed response by the client with UnexpectedPacket (E2, A15/I1), that an initiation with a forged mac1 is refused with InvalidMac (E4), and that the client processes a cookie reply correctly: the retransmitted initiation after a cookie carries a valid mac2 (A14).

Counter and replay window. boringtun's session.rs implements a 1,024-bit sliding window (`N_WORDS = 16`, session.rs:36). Through the crate's `WgPeer::decapsulate` the team confirmed in-order acceptance, DuplicateCounter on replay, acceptance of a late packet inside the window, InvalidCounter 1,147 packets behind, InvalidAeadTag on a flipped ciphertext bit and on trailing bytes, NoCurrentSession for a foreign receiver index and InvalidPacket for a truncated message (A10/E3), and the same outcomes when the records travel over the live carrier (A15).

Carrier framing robustness. record.rs (encode, parse_header, RecordParser) was read in full and then exercised with sixteen hand-built headers, batching and trickle timing, 64 MiB of random garbage, a 20-second mutation fuzz of ten million records and, over the live carrier, replayed, flipped, oversized, misframed and fragment-flagged records plus a mid-record close (A11, A15).

Endpoint and roaming authentication. On the client the only authenticated element is the Noise handshake; the TCP endpoint is whatever the configuration says. On the in-tree server, a carrier is adopted as a peer's path only after an inner packet decrypted under that peer's session and passed the source-address check (engine.rs:137-148, run.rs:188-194, peers.rs:48-71), and a stale generation cannot displace a newer path. These paths were read and are covered by the crate's own tests (engine.rs:290-303, peers.rs:305-361), which passed (A2).

Configuration parsing and secret handling. config.rs, lib.rs and main.rs were reviewed for how keys enter, where they are copied, what derives Debug or Serialize and what is logged. The loader was fed oversized, truncated, nested, mistyped, misspelled and degenerate inputs, and `run_tunnel` was called with unvalidated configurations (A9). The CLI was run with a world-readable configuration against a dead endpoint and its log inspected (A19).

Error handling and half-open tunnels. carrier.rs was read for every path that leaves the event loop. Three live experiments were run: a listener that accepts and discards everything (A12), a responder that holds the

connection open but stops reading (A16), and a responder that closes the carrier in the middle of a record (A15/I8).

Resource bounds. Buffer sizes and queue depths were enumerated (carrier read buffer 64 KiB, carrier.rs:139; boringtun scratch 65,567 bytes, wg.rs:14; per-stream smoltcp buffers 2 x 256 KiB, netstack.rs:17-18; per-stream channels 2 x 64, carrier.rs:226-227; connect queue 128, lib.rs:57; boringtun packet queue 256, vendor mod.rs:40; parser maximum record 65,543 bytes, record.rs:23). Resident memory was sampled under 500 idle SOCKS5 connections, 200 held CONNECTs, the garbage flood and the mutation fuzz (A12, A11).

Unsafe code and FFI surface. `grep -rn unsafe src examples` returns nothing. The library exposes no C ABI; the only foreign-looking surface is the socket-protection hook `fn(i32)` (carrier.rs:338-343) that an embedder installs. Unsafe counts in the vendored crates on the client's path are tabulated in Appendix B.

Dependency hygiene and vendoring. cargo audit 0.22.2 against the RustSec database as of 2026-10-14, `cargo tree -d`, the lockfile, .cargo/config.toml and the vendor directory were examined (A6, A7).

Timers, rekey and keepalive. A tunnel with a small write every 5 s was observed for 140 s and one with a single initial exchange for 215 s, recording every record's type, length and time (A17, A18). The discard listener recorded the initiation retry cadence and the 90 s expiry (A12). Of the protocol limits named in the brief, REKEY_AFTER_TIME (120 s), REKEY_ATTEMPT_TIME (90 s), REKEY_TIMEOUT (5 s) and the persistent keepalive were observed directly; the REJECT_AFTER_TIME expiry (180 s) was not reached because the 120 s rekey pre-empts it once data has been sent, and REKEY_AFTER_MESSAGES (2^60) is unobservable by magnitude and, as noted in RVPN-01-515, not implemented by boringtun.

Carrier fingerprint. The first bytes of every carrier connection were captured by the discard listener and by the responder, in both directions (A12, A13).

SOCKS5 front-end. Method negotiation, SOCKS4, zero methods, zero-length domain, truncated request, idle flood, host-name target and closed inner port were exercised against the live listener (A13).

Dynamic tests executed

Twenty-one test runs were executed on the assessment host (macOS 25.6.0 on Apple Silicon; rustc 1.91.1; cargo 1.91.1; clippy 0.1.91; cargo-audit 0.22.2 with RustSec advisory database commit edb58f59 of 2026-10-14, 1,329 advisories; Python 3.15.0; pandoc 3.8.2.1). All services bound 127.0.0.1 within ports 20500 to 20599. Scratch work lived under /tmp/rvpn01/wp5; the repository was reached through a symbolic link at /tmp/rvpn01/wp5/repo and every cargo invocation used CARGO_TARGET_DIR=/tmp/rvpn01/wp5/target with –locked, so the tree itself was never written to (at 05b491a, checksums of all non-vendor files were taken before and after and are identical, A20; the re-verification tree is covered by B17). The harness is a scratch crate of about 1,400 lines (wp5harness) that depends on the library by path and builds against the repository's vendored sources. Each live scenario ran in its own process because `carrier_is_up()` and the protection hook are process-global statics in carrier.rs. The crate's own unit tests bind 127.0.0.1:0 (ephemeral ports chosen by the target's test code, outside the assigned block); this is noted in A2.

Id	Test	Result
A1	cargo build –offline –locked	pass, 128 crates compiled
A2	cargo test –offline –locked	pass, 84 tests, 0 failed
A3	cargo clippy –offline –locked –all-targets	pass, 0 warnings
A4	examples build; genkey example run	pass, two 44-character base64 lines
A5	CLI genkey, pubkey, help; server help	pass
A6	cargo audit –no-fetch	0 vulnerabilities, 1 yanked crate
A7	cargo tree –offline –locked -d	9 crates present in two versions
A8	harness build (release)	pass
A9	config-fuzz (C1 to C7)	2 panics on unvalidated input, 3 validation gaps

Id	Test	Result
A10	engine-cost (E1 to E5)	100x CPU asymmetry; replay checks pass
A11	record-fuzz (R1 to R5)	no panic, bounded memory, quadratic trickle
A12	discard listener, 100 s	flag true without handshake; expiry at 90.0 s
A13	tunnel baseline + SOCKS battery	1,114 Mbit/s echo; 6 SOCKS deviations
A14	tunnel cookie	cookie handled, mac2 valid at +5.002 s
A15	tunnel replay (I1 to I8)	all hostile records rejected; recovery 1.0 s
A16	tunnel stall, 120 s zero window	loop stalled, flag true, CONNECT hangs
A17	tunnel idle, 215 s	keepalive 25.00 s; rekey at 120.1 s
A18	tunnel rekey, 140 s	rekey at 120.0 s after session
A19	CLI run, 0644 config, dead endpoint	both public keys logged at info
A20	tree checksum before and after	identical
A21	pandoc render of this report	parses

Re-verification for version 1.1. The same harness, adjusted for the new `NetStack::new` signature and the trimmed tokio feature set, was run against the working tree at HEAD 27a912d on 2026-10-15 (06:39Z to 06:46Z): B1 build, embedder build with `--no-default-features`, 100 + 2 unit tests, clippy and the stripped release build; B2 audit; B3 duplicate crates; B4 harness rebuild; B5 config-fuzz; B6 engine-cost; B7 record-fuzz; B8 discard listener with the stream-cap check; B9 baseline with the family-mismatch check; B10 cookie; B11 replay; B12 stall against the write deadline; B13 idle; B14 rekey; B15 shutdown; B16 CLI; B17 tree checksum; B18 render. Eighteen entries (B1 to B18) with 155 recorded checks, all on loopback within the assigned block; they are not counted among the twenty-one runs of the original campaign. Details in Appendix A.

Standards and specifications applied

OWASP ASVS 5.0.0 (30 May 2025) as the control framework for a library (Appendix C)[1]; CWE for every finding and CVSS v4.0 (FIRST, November 2023) vector strings; NIST SP 800-115 for the test process; NIST SP 800-218 SSDF v1.1, SLSA v1.0, CycloneDX 1.7 and SPDX 3.0.1 for supply-chain and SBOM observations; Regulation (EU) 2024/2847 (Cyber Resilience Act) for the reporting and vulnerability-handling obligations that will apply to products placed on the EU market; the Noise Protocol Framework revision 34 and the WireGuard protocol as published in the whitepaper and at wireguard.com/protocol for the handshake, counters, timers and cookies[2]; RFC 9293 for the TCP carrier; RFC 1928 and RFC 1929 for SOCKS5; RFC 5737 and RFC 3849 for documentation addresses.

Identified Vulnerabilities

RVPN-01-501 WP5: Carrier identifiable from its first sixteen bytes: plaintext record header, deterministic session index, fixed sizes and timers (High)

Fix Note: Partially fixed by the development team in commit c0fdb7d (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d: the session index is now random per peer instance,

[1] OWASP Application Security Verification Standard 5.0.0, chapters V1 to V17, 30 May 2025.

[2] J. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel", sections 5 and 6 (handshake, cookies, timers, counters); the protocol page lists REKEY_AFTER_MESSAGES = 2^60, REJECT_AFTER_MESSAGES = 2^64 - 2^13 - 1, REKEY_AFTER_TIME = 120 s, REJECT_AFTER_TIME = 180 s, REKEY_ATTEMPT_TIME = 90 s, REKEY_TIMEOUT = 5 s, KEEPALIVE_TIMEOUT = 10 s.

masked to 24 bits (wg.rs:16-18, :79-88), and each timer tick carries 0 to 333 ms of jitter (carrier.rs:28-30, :198-200, :357-364). On the wire (B8, B9) the constant prefix is thirteen bytes instead of sixteen, `00 00 00 9c 00 00 b6 25 01 00 00 00 01`, because boringtun keeps the low index byte as a counter starting at 1; the 24 random bits stay fixed for the peer's lifetime, so every reconnect of one running client repeats bytes 13 to 15, a linkability property kernel WireGuard avoids by drawing a fresh index per handshake; retries arrive 5.03 to 5.48 s apart. Framing, sizes and cadence are unchanged, so the finding and its severity stand.

The WGTCP carrier frames every WireGuard message in an 8-byte header: a big-endian length, a type byte of 0, a flags byte and a 16-bit checksum that is a public function of the other three (record.rs:26-34, README.md:64-66). The WireGuard message that follows carries type, reserved zeros, indices and counter in clear. Nothing wraps the stream. The first write on every connection is the handshake initiation (carrier.rs:134-137). At 05b491a `WgPeer::new` passed `index = 0` to boringtun (wg.rs:72), which derives the session index as `(index << 8)| counter` (vendor/boringtun/src/noise/handshake.rs:466-471), so bytes 12 to 15 read 1 on the first initiation of every process and counted up from there, instead of being random as the protocol intends.

Affected: src/record.rs:26-56 (`header_checksum`, `encode`); src/carrier.rs:134-137 (first write); src/wg.rs:66-75 (index 0); src/config.rs:19-21 (keepalive); vendor/boringtun/src/noise/timers.rs:22, :228-236 (retransmission). Also affects WP6.

Proof of Concept: The first read on every connection (A12, A13) was:

```
carrier #1: first read 156 B; bytes 0..16 = 00 00 00 9c 00 00 b6 25 01 00 00 00 01 00 00 00
record 1 at t=0.002s: wg_type=1 msg_len=148 sender_idx=1 mac2_zero=true
record 2 at t=5.003s: wg_type=1 msg_len=148 sender_idx=2 mac2_zero=true
carrier #2: first read 156 B at t=90.513s; bytes 0..16 = 00 00 00 9c 00 00 b6 25 01 00 00 00 13 00 00 00
server first bytes (handshake response record): 100 B: 00 00 00 64 00 00 a9 25 02 00 00 00 01 01 00 00
```

Cookie replies (72 bytes), keepalives (40 bytes every 25.00 s, A17), the rekey pair (156 + 100 bytes at 120.0 s, A18) and data records (inner packet plus 40 bytes, unpadded) are constant too.

Impact: A national DPI operator can block the service with a single content rule on the first segment of a TCP connection, with no false positives, and can enumerate users by logging the matches. The threat model rates fingerprinting that identifies users as High unless mitigated, and nothing in the tree mitigates it; port 443 does not help because the payload is not a TLS ClientHello.

Preconditions: Passive observation of the carrier stream; no keys.

Recommendation: Carry the record stream inside a cover protocol the kernel module can share unchanged: TLS 1.3 with a browser-like ClientHello and random record padding, or a keyed framing in the Shadowsocks-2022 family with the length AEAD-encrypted under a key derived from the Noise static-static DH or the PSK and records padded to random lengths. Independently, draw a fresh index per handshake and precede the initiation with a random-length preamble the server skips with its resynchronising parser. Header changes must be mirrored in the kernel module (WP6).

Classification: CWE-203 · OWASP ASVS 5.0.0 V12 Secure Communication · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N

RVPN-01-502 WP5: carrier_is_up() is raised on TCP connect and stays true for 90 s of every retry cycle without a handshake (Medium)

Fix Note: This issue has been fixed by the development team in commits 22dba80 and 9215cdd (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d to be working as expected: `CARRIER_UP` is now raised only when `WgPeer::handshakes()` has advanced on the current connection, which happens solely on a handshake response that boringtun accepted (carrier.rs:150, :211-214; wg.rs:128-152), and a `Readiness` guard clears it whenever `serve` returns or its task is dropped (carrier.rs:141-147, :384-390). Against the accept-and-discard listener the flag stayed false for the whole 100 s run (B8); against the responder it rose only after the log line `wireguard handshake completed` (B9); a replayed handshake response did not count (B11); carrier loss cleared it (B11, B12); and a shutdown request cleared it within 22 ms (B15).

`Carrier::run` stores `CARRIER_UP = true` the moment `connect_carrier()` returns a connected socket (carrier.rs:97-100); the handshake initiation is written afterwards inside `serve()` and its completion is never consulted

(carrier.rs:134-137). The flag is cleared only when `serve()` returns (carrier.rs:105), which against a silent peer happens when boringtun's REKEY_ATTEMPT_TIME expires after 90 s (vendor/boringtun/src/noise/timers.rs:217-226, surfaced through wg.rs:141-143 and carrier.rs:182). The library already has the right signal, `WgPeer::time_since_last_handshake()` (wg.rs:151-153), but nothing reads it.

Affected: src/carrier.rs:97-106 (`Carrier::run`), :329-335 (`carrier_is_up`); src/wg.rs:151-153.

Proof of Concept: Test A12 ran the client against a listener that accepts the TCP connection and discards every byte. The flag sampled every 250 ms and the library's own log read:

```
carrier established to 127.0.0.1:20500 t=0.002s
carrier_is_up() -> true at t=0.253s
(initiations resent at 5.0 s intervals, no response ever sent)
carrier lost: wireguard timers: connection expired t=90.010s
carrier_is_up() -> false at t=90.243s
carrier established to 127.0.0.1:20500 t=90.513s
carrier_is_up() -> true at t=90.746s
```

The flag read true for 89.99 s of every 90.5 s cycle, indefinitely, with zero handshakes completed and zero bytes of application data carried. Test A14 shows the same race in the benign case: the flag became true at t=0.456 s while the handshake only completed at t=5.206 s after the cookie exchange.

Impact: An embedder that drives its user interface or its transport fallback from this flag shows "connected" against any middlebox that completes TCP and drops payload, and against a server that has revoked the key (the server is silent by design, engine.rs:83-86). No traffic leaves the device unprotected inside this library, so the item is not the Critical silent-failure case, but the user believes they are protected while every request merely times out after 30 s (netstack.rs:182), and a fallback to another transport that is gated on this flag never triggers. Also affects WP3 and WP4, which consume the flag.

Preconditions: A peer or middlebox that accepts the TCP connection and stays silent. No keys.

Recommendation: Define readiness as "a handshake has completed on the current carrier". In `serve()`, after each `decapsulate` and `update_timers`, set the flag when `self.peer.time_since_last_handshake()` first returns `Some` after the connect; clear it on `ConnectionExpired` and on carrier loss. Export a three-state value (`Connecting`, `Up`, `Down`) and the handshake age so embedders can show "reconnecting" honestly, and add a unit test with an accept-and-discard listener that asserts the flag never becomes true.

Classification: CWE-392 · OWASP ASVS 5.0.0 V16 Security Logging and Error Handling · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:

RVPN-01-503 WP5: SOCKS5 host names and the carrier endpoint name are resolved by the host resolver outside the tunnel (Medium)

The tunnel carries only TCP (README.md:128-137), so there is no resolver inside it. A SOCKS5 CONNECT with ATYP 0x03 is resolved with `tokio::net::lookup_host` on the physical network before the connection enters the tunnel (socks5.rs:121-127, :165-181); the one address chosen is then dialled through the tunnel. If `endpoint` is a host name it is resolved the same way on every reconnect (carrier.rs:348 and :351). The README documents the limitation (README.md:222-227) and at the same time documents `curl --socks5-hostname` as the way to use the client (README.md:103), which is precisely the leaking form.

Affected: src/socks5.rs:121-127 (`handle`), :165-181 (`resolve`); src/carrier.rs:346-354 (`dial`).

Proof of Concept: Test A13 issued a CONNECT for the name "localhost", which the host resolver answers from its hosts file so that no query left the assessment host; the responder decrypted the resulting inner SYN:

```
D1 CONNECT ATYP=domain "localhost":9 -> reply code: 0
D1 inner SYN destinations seen by the responder: [(127.0.0.1, 9)]
```

The name was resolved by the host and only the resulting address entered the tunnel. On a real network the same code path emits a plaintext DNS query for every host name an application hands to the proxy. The code excerpt:

```
// socks5.rs:166-170
async fn resolve(host: &str, port: u16) -> Option<IpAddr> {
   if let Ok(ip) = host.parse::<IpAddr>() { return Some(ip); }
   let addrs: Vec<SocketAddr> = tokio::net::lookup_host((host, port)).await.ok()?.collect();
```

Impact: While the tunnel is up and reported as such, the local network and the censor see a time-stamped list of the host names the user visits and can block or redirect by DNS alone; a poisoned answer becomes a connection to the wrong host inside the tunnel. If the server endpoint is configured by name, the server's name is exposed on every reconnect.

Preconditions: An application that sends names to the proxy (the documented usage) or an endpoint configured as a host name.

Recommendation: Resolve inside the tunnel: the TCP-only stack can already carry DNS over TCP (RFC 7766) or DNS over TLS to a resolver reached through smoltcp, and inner UDP would allow plain DNS to the gateway. Until then fail loudly rather than silently: add a `resolve_domains = "refuse" | "system"` setting defaulting to `refuse` that answers ATYP 0x03 with reply 0x08, require `endpoint` to be an IP literal, and change README.md:103 to `--socks5` with an address.

Classification: CWE-319 · OWASP ASVS 5.0.0 V12 Secure Communication · CVSS:4.0/AV:A/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N

RVPN-01-504 WP5: Unauthenticated loopback SOCKS5 proxy with no connection cap and no request timeout, usable by any co-resident process (Medium)

Fix Note: Partially fixed by the development team in commit 7fc5231 (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d: a greeting that does not offer method 0x00 is answered with 05 ff and closed (socks5.rs:147-152; B9/S1, B8/S3), the greeting and request phases are bounded by a 10 s negotiation timeout (socks5.rs:35, :87-90; B9/S5 closed after 10.0 s), concurrent streams are capped by a 512-permit semaphore with reply 0x01 beyond it (socks5.rs:32, :58, :103-106; in B8 the cap engaged at the 512th concurrent stream, 511 held by the harness plus one still in teardown), a non-loopback bind logs a warning (socks5.rs:54-56) and accept errors back off (socks5.rs:37-39, :65-66). What remains: the listener is still unauthenticated, a non-loopback bind is still permitted, and the per-stream buffers are still allocated eagerly. The caps are constants, `MAX_STREAMS = 512` and `NEGOTIATION_TIMEOUT = 10 s` in socks5.rs:32-35, not configuration.

The SOCKS5 listener accepts every connection and spawns a task per connection with no limit (socks5.rs:35-56). It answers every greeting with method 0x00 regardless of what the client offered (socks5.rs:72); it never times out while waiting for a request; and each successful CONNECT allocates a smoltcp socket with two 256 KiB buffers and two 64-slot channels (netstack.rs:178-180, carrier.rs:226-227). The CLI's `--socks` override binds wherever it is told with no loopback check (main.rs:36-38, :83-85). The threat model names co-resident malicious applications; on Android and on desktops loopback is reachable from every process on the device.

Affected: src/socks5.rs:35-72 (`serve`, `handle`); src/netstack.rs:177-196 (`NetStack::connect`); src/main.rs:81-85 (`run`).

Proof of Concept: From test A13 and A12 against the live listener:

```
S1 greeting offering only method 0x02 -> reply: "05 00" (RFC 1928 expects 05 ff)
S5 truncated CONNECT then 3 s silence -> server: still waiting (no timeout on request read)
S6 500 idle connections to the SOCKS5 port: 500 open, rss 3.8 -> 4.4 MiB; fresh CONNECT still answered
S7 200 SOCKS5 CONNECTs held open: 200 replied success in 1.05 s; rss 4.6 -> 9.1 MiB (23 KiB per stream)
```

Every connection from an unprivileged local process was accepted and tunnelled under the user's server identity. The measured resident cost per idle stream is 23 KiB, not the 512 KiB the buffer sizes suggest, because the zero-filled buffers are not touched until data flows; memory exhaustion therefore requires traffic, while file descriptors (two per stream plus the carrier) are the first resource to run out.

Impact: Any co-resident application can detect that the VPN is running, route its own traffic through the user's tunnel so that it is attributed to the user's key and consumes the user's quota, or exhaust the process's file

descriptors and stall the tunnel. An operator who binds to a non-loopback address obtains an open proxy on the local network without a warning. Also affects WP3, where the listener is reachable by every installed app.

Preconditions: A local process on the same host (or on the same network if `--socks` is pointed at a non-loopback address).

Recommendation: Implement method selection per RFC 1928 and offer RFC 1929 username/password with a per-launch random secret that the embedder passes to its own networking stack; refuse non-loopback binds unless an explicit `allow_remote = true` is set; cap concurrent streams (reply 0x01 beyond the cap) and total accepted connections; apply a read timeout (for example 10 s) to the greeting and request phases; and allocate the smoltcp buffers lazily or at 64 KiB, which is ample at a 1,380-byte MTU.

Classification: CWE-306 · OWASP ASVS 5.0.0 V6 Authentication · CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:N/VI:L/VA:L/SC:L

RVPN-01-506 WP5: Private key printed by Config's derived Debug and Serialize, never zeroised, read from a file whose permissions are not checked (Low)

Fix Note: Partially fixed by the development team in commit fcce688 (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d: `Config` now has a hand-written `Debug` that prints `<redacted>` for the private and pre-shared keys (config.rs:72-91; B5/C1 reports the key absent), and `Config::load` warns when the file is readable by group or others (config.rs:93-112; B16 logged `config config.toml is readable by other users (mode 644)`). What remains: `Serialize` is still derived and `toml::to_string` still re-emits the key (B5/C1), the `[u8; 32]` and `String` copies are still not zeroised, and the permission check warns rather than refuses.

`Config` holds the private key as a `String` and derives `Debug`, `Clone` and `Serialize` (config.rs:23-26). Any `{:?}` of the configuration, for example in an embedder's error context, crash reporter or debug log, prints the key verbatim, and any serialisation re-emits it. `Config::load` reads the TOML file with `read_to_string` without inspecting its mode (config.rs:68-74). The key is copied into `[u8; 32]` arrays in `run_tunnel` (lib.rs:26), `main::run` (main.rs:87) and `WgPeer::new` (wg.rs:63), and `generate_keypair` returns it as a base64 `String` (lib.rs:81-90); none of these copies is zeroised when dropped. The vendored x25519-dalek does zeroise `StaticSecret` on drop (vendor/x25519-dalek/src/x25519.rs:73-74, feature enabled by default), so boringtun's own copy is covered; the crate's copies are not.

Affected: src/config.rs:23-26, :68-74 (`Config`, `Config::load`); src/lib.rs:26, :81-90; src/main.rs:67, :87; src/wg.rs:57-63.

Proof of Concept: Test A9 built a configuration with a throwaway key and formatted it:

```
C1 Config Debug output contains the private key verbatim: true
C1 Config Debug output length: 311
C1 toml::to_string(&Config) re-emits the private key: true
```

Test A19 wrote a configuration with the default umask (mode -rw-r–r–) and ran `redsocket-wgtcp run --config config.toml`; the binary loaded it without any warning. The private key did not appear in the log (0 matches), which is correct; the key was found only through the Debug and Serialize paths above.

Impact: On a seized device, or in a crash report or log collector fed by an embedder that formats its configuration, the user's long-term private key is exposed; whoever holds it can impersonate the user to the server. Key bytes that are not zeroised can persist in freed memory, core dumps and swap. A world-readable configuration lets another local account read the key.

Preconditions: An embedder or operator that formats or serialises the configuration, a memory dump, or a second local account on a desktop deployment.

Recommendation: Store secrets in `zeroize::Zeroizing<String>` or a dedicated `SecretKey` newtype with a redacting `Debug` (`zeroize` 1.9.0 is already in the dependency graph); drop `Serialize` from `Config` or mark the secret fields `#[serde(skip_serializing)]`; zeroise the `[u8; 32]` copies after `Tunn::new`; refuse (or at least warn on) a configuration file with `mode & 0o077 != 0`, as wg-quick does; and document that embedders must inject keys from platform key storage rather than files.

Classification: CWE-312 · OWASP ASVS 5.0.0 V13 Configuration (V13.3 Secret Management) · CVSS:4.0/AV:L/AC:L/AT:P/PR

RVPN-01-511 WP5: A carrier peer that stops reading stalls the event loop indefinitely: WireGuard watchdog silenced, tunnel reported up, new connections hang (High)

Fix Note: Partially fixed by the development team in commit 5c51009 (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d: every carrier write now runs under a 50 s deadline (carrier.rs:40-43, :366-372) whose expiry counts as carrier loss. In B12 the client logged `carrier lost: carrier write: peer stopped reading for 50s` 49.98 s after the responder stopped reading, cleared the flag, reconnected within 0.5 s, and answered a CONNECT issued during the stall after 35.5 s instead of never. The stall is bounded, not eliminated: for up to 50 s per occurrence the loop is still blocked (reads, timers and connect requests pause, the flag reads true), and a peer that stalls after every reconnect re-triggers it; the restructuring below remains open and the heading severity is kept for the assessed commit.

At 05b491a `Carrier::serve` was one `tokio::select!` loop that read the carrier, accepted connect requests, drove the WireGuard timers and polled the stack (carrier.rs:149-187), then wrote the batch inline with `write_all` (carrier.rs:201-210). With the peer's window closed the write never completed and the loop stopped: no reads, no timer ticks, no connect requests. The WireGuard expiry is evaluated only inside `update_timers` (vendor/boringtun/src/noise/timers.rs:168-226), so it never fired; `CARRIER_UP` stayed true because `serve()` had not returned, and the SOCKS5 tasks blocked on the connect channel (socks5.rs:130-137).

Affected: src/carrier.rs:149-213 (`Carrier::serve`, inline `write_all` at :209, timers at :178-184, connect requests at :174-176); src/wg.rs:137-147.

Proof of Concept: Test A16 pushed data over eight streams through the loopback responder, which after 8 s kept the TCP connection open but stopped reading (receive buffer 32 KiB); the harness sampled the flag and the kernel socket queues:

```
responder stopped reading the carrier at t: 8.517s
stall +5s: carrier_is_up()=true netstat: Recv-Q 430560 (responder) | Send-Q 654892 (client) ESTABLISHED
stall +15s: new SOCKS5 CONNECT (30 s budget): NO REPLY within 30 s
stall +120s: carrier_is_up()=true (queues unchanged)
closing the stalled carrier with RST at t: 128.614s
carrier lost: carrier write: Broken pipe (os error 32)
carrier lost: wireguard timers: connection expired (only now, on the next tick)
```

For the full 120 s the log had no "carrier lost" or "connection expired" line; both appeared only on RST. Against a silent peer (A12) the same client gives up at 90.01 s: only the blocked write defeats the watchdog.

Impact: A transparent TCP middlebox that passes the handshake and then withholds acknowledgements with a zero window, or any path stall of that shape, froze the client with the tunnel reported as up: open streams hung, new CONNECTs got no reply, no reconnect or fallback happened, and the embedder had no signal; the censor needs no key and causes no visible block. Where the peer stops answering altogether, recovery depends on the host TCP retransmission limit, typically minutes, rather than WireGuard's 90 s; that case could not be induced on the assessment host.

Preconditions: An on-path position or a stalled path after TCP establishment; no keys.

Recommendation: Move writes to a dedicated task fed by a bounded channel (the in-tree server does this, run.rs:83-100), treat a full channel as back-pressure, and put a deadline on the write path whose expiry counts as carrier loss, for example:

```
// carrier.rs, in serve(): replace the inline write_all with
tokio::time::timeout(WRITE_DEADLINE, wr.write_all(&batch)).await
   .map_err(|_| anyhow!("carrier write stalled for {WRITE_DEADLINE:?}"))?
   .context("carrier write")?;
```

with the deadline around 2 x `persistent_keepalive`; keep `update_timers` on its own task, set `TCP_USER_TIMEOUT` on Linux and Android, and add a test with a non-reading peer.

Classification: CWE-1088 · OWASP ASVS 5.0.0 V15 Secure Coding and Architecture · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/

RVPN-01-512 WP5: Userspace server performs X25519 on every unauthenticated initiation before checking mac1, and its cookie defence can never engage (Low)

The in-tree server identifies the peer behind an initiation with `parse_handshake_anon` (engine.rs:71-86), which performs an X25519 Diffie-Hellman and two AEAD operations (vendor/boringtun/src/noise/handshake.rs:348-363) before any message authentication: mac1 is only verified later, inside `Tunn::decapsulate` (vendor/boringtun/src/noise/rate_limite 170). The engine then calls `decapsulate(None, ...)` with no source address (engine.rs:130), and boringtun's rate limiter, once it is under load, returns `UnderLoad` instead of a cookie reply whenever the address is absent (rate_limiter.rs:172-176). The WireGuard cookie mechanism, the protocol's defence against exactly this computational DoS, is therefore unreachable on this server; the client side handles cookies correctly (A14).

Affected: src/server/engine.rs:70-101 (`Engine::identify`), :103-130 (`Engine::ingress`); vendor/boringtun/src/noise/rate_limiter.r 192 as consumed.

Proof of Concept: Test A10 timed `Engine::ingress` in-process on the assessment host (one core, release build):

```
E1 valid-mac1 initiation from an unregistered key: 32.2 us/msg -> 31,026 msg/s/core (all: handshake from
    ↪ unregistered peer)
E1 148 random bytes, type=1 (garbage mac1): 31.7 us/msg -> 31,557 msg/s/core (all: anonymous handshake
    ↪ parse failed)
E1 148 random bytes, type=9: 0.3 us/msg -> 3,032,794 msg/s/core
E1 148 random bytes, type=4 (unknown index): 0.2 us/msg -> 4,222,750 msg/s/core
```

A 148-byte message whose only property is a leading `01 00 00 00` costs the server a hundred times more than any other garbage, and the cost is paid before any check that the sender knows anything. In test A10/E4 the same limiter on the client side dropped 42 of 50 valid-mac1 initiations within one second with `UnderLoad`, demonstrating the no-address branch.

Impact: About 31,000 forged initiations per second, or roughly 37 Mbit/s of 148-byte records over a handful of TCP connections, saturate one server core; legitimate handshakes are then dropped as `UnderLoad` rather than challenged with a cookie. The README states that production servers run the kernel module, so the exposure is limited to deployments of redsocket-wgtcp-server; it would be Medium if that binary were placed at the edge.

Preconditions: Network reach to a listener of the userspace server. No keys.

Recommendation: Verify mac1 before `parse_handshake_anon`: compute keyed BLAKE2s-128 over the first 116 bytes with the key `BLAKE2s-256("mac1----" || server_public)` and compare in constant time (the same primitive boringtun uses). Pass the carrier's remote address as `src_addr` to `decapsulate` so that the limiter can issue cookie replies, and share a single `RateLimiter` across all peers through the `rate_limiter` argument of `Tunn::new` rather than the per-peer default. Add per-source-address connection limits on the TCP listener.

Classification: CWE-400 · OWASP ASVS 5.0.0 V2 Validation and Business Logic · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N

Miscellaneous Issues

RVPN-01-505 WP5: Destinations, host names, the endpoint and both static public keys written to the log (Low)

Fix Note: This issue has been fixed by the development team in commit 1d930aa (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d to be working as expected: the CLI logs both static keys as 8-character prefixes through `wg::key_label` (main.rs:79-84, wg.rs:44-46), the carrier names the endpoint only at debug level (`dialing`, carrier.rs:131) and no longer in the established, lost or retry lines (carrier.rs:110-118), the SOCKS5 success path no longer logs the destination (socks5.rs:129-132) and its error texts name neither host nor address (socks5.rs:98, :105), and the server configuration no longer derives `Debug` (bin/server.rs:21, :58). In the CLI run B16 at the default level the full peer public key was absent (0 matches, prefix only), and in B9 at debug level no destination line appeared.

At the default `info` level the CLI logs the client's own public key, the server's public key and the endpoint (main.rs:88-96), and the carrier logs the endpoint on every connect, loss and retry (carrier.rs:99, :103, :108, :124). At `debug` the SOCKS5 front logs every destination it tunnels (socks5.rs:152) and includes unresolvable

host names in error text (socks5.rs:125, :52). `RUST_LOG` is honoured in release builds (main.rs:48, bin/server.rs:97). The library writes no files, but embedders and service managers capture stderr.

Affected: src/main.rs:88-96 (`run`); src/carrier.rs:99-124; src/socks5.rs:52, :125, :152.

Proof of Concept: Test A19 ran the CLI at the default level against a dead endpoint (keys shortened here):

```
INFO redsocket_wgtcp] our public key: <key...HEk=>
INFO redsocket_wgtcp] peer <key...0nk=> via TCP carrier 127.0.0.1:20599
```

The full peer public key was present in the captured stderr (1 match); the private key was not (0 matches). In test A13 at `debug` level every CONNECT produced `tunneling to 10.50.0.1:8081` with the real destination.

Impact: A seized device or a captured log yields a durable identifier of the user (the static public key), the server identity, and at debug level a time-stamped list of destinations.

Preconditions: Access to the log output; debug level for destinations.

Recommendation: Log keys only as an 8-character prefix as the server already does (peers.rs:42-45); never log destination addresses or host names at any level (log counts and durations instead); log the endpoint only at debug; and document the stderr policy for embedders.

Classification: CWE-532 · OWASP ASVS 5.0.0 V16 Security Logging and Error Handling · CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:N

RVPN-01-507 WP5: Dependency hygiene: yanked chacha20 0.10.1 via rand, duplicated crates, tokio full, tun in the library graph, no toolchain pin, no release profile (Low)

Fix Note: Partially fixed by the development team in commits c0fe855 and 85ac239 (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d: `rand` and with it the yanked chacha20 0.10.1 are gone and `genkey` calls `generate_keypair()` (main.rs:57-62), cargo audit reports 0 vulnerabilities and 0 warnings (B2), the lockfile shrank from 172 to 166 packages and six vendored crates were removed (B3), `tun` is optional behind the default `server` feature and the server binary requires it (Cargo.toml:24-39), tokio is reduced to six features (Cargo.toml:22), and `[profile.release] strip = true` cuts the release binary from 16.3 MB to 3.1 MB (B1). What remains: no toolchain pin, no `lto`, `codegen-units` or `panic` setting, no SBOM, no CI with cargo audit or cargo deny, and six crates still in two versions (Appendix B). Embedders build the library with `default-features = false` to leave out the server binary and the TUN crate.

The lockfile resolves `rand 0.10.2` to `chacha20 0.10.1`, which has been yanked from crates.io; the only consumer of `rand` is the CLI's `genkey` (main.rs:9, :60), which duplicates `generate_keypair()` in lib.rs:81-90 (that one already uses `getrandom::fill`). The graph carries two versions each of nine crates (base64, bitflags, chacha20, cpufeatures, defmt, getrandom, rand_core, syn, windows-sys); `tokio` is pulled with `features = ["full"]` (Cargo.toml:23); the `tun` crate, needed only by the server binary, is a plain `[dependencies]` entry and therefore compiled for every target including the library the mobile apps vendor (Cargo.toml:25); there is no rust-toolchain.toml and no `[profile.release]` (so release builds keep `panic = "unwind"` and full symbols unless the embedder strips them). The positive side is strong: all 171 external packages come from crates.io with checksums, 0 git or path sources, the sources are vendored and `--remap-path-prefix` plus `-ffile-prefix-map` scrub build paths (.cargo/config.toml:2, :11).

Affected: Cargo.toml:8-25; Cargo.lock (rand 0.10.2 -> chacha20 0.10.1); src/main.rs:58-70 (`genkey`).

Proof of Concept: Test A6, `cargo audit --no-fetch` (1,329 advisories, database of 2026-10-14):

```
Scanning Cargo.lock for vulnerabilities (172 crate dependencies)
Crate: chacha20 Version: 0.10.1 Warning: yanked
warning: 1 allowed warning found
```

Test A7, `cargo tree --offline --locked -d`, listed five of the pairs for the host target; the other four are visible in the vendor directory.

Impact: A yanked release is one its maintainers withdrew; a clean re-resolution without the vendored copy fails or silently changes version; two key-generation paths invite drift; the unpinned toolchain weakens reproducibility; the extra crates enlarge the attack surface and the binary that ships inside the apps.

Preconditions: None (supply-chain posture).

Recommendation: Delete the `rand` dependency and have `genkey` call `redsocket_wgtcp::generate_keypair()`; move `tun` under a server-only feature (`[features] server = ["dep:tun"]`) or a workspace member; trim tokio features to `rt-multi-thread`, `net`, `io-util`, `sync`, `time`, `macros`; add rust-toolchain.toml, `cargo audit` and `cargo deny` to CI; add a `[profile.release]` with `lto = true`, `codegen-units = 1`, `strip = true` and `panic = "abort"` for the CLI; and publish a CycloneDX 1.7 or SPDX 3.0.1 SBOM from the lockfile (cargo-cyclonedx) so that downstream packaging under the EU Cyber Resilience Act has a machine-readable inventory.

Classification: CWE-1104 · OWASP ASVS 5.0.0 V15 Secure Coding and Architecture (V15.2 Security Architecture and Dependencies) · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N

RVPN-01-508 WP5: Record parser copies and rescans its whole buffer on every call (Info)

Fix Note: This issue has been fixed by the development team in commit c612da1 (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d to be working as expected: the parser now works on `VecDeque::make_contiguous` and copies only the payload out (record.rs:150-185). In B7 the 1,600-record batch took 40.8 us instead of 1.66 ms, the byte-trickled maximum record 303 us instead of 28.7 ms, and the 20-second mutation fuzz processed 12.7 million records with no panic and a maximum resident set of 4.7 MiB.

`RecordParser::next_message` calls `materialize()` at the top of every iteration, copying the entire `VecDeque` into a contiguous scratch vector (record.rs:160, :195-201), and on a header miss `find_resync_offset` tests every offset (record.rs:204-210). Memory is bounded: a header cannot claim more than 65,543 bytes (record.rs:102) and on a miss at most 7 bytes are kept (record.rs:180-185). The cost is CPU, quadratic in the buffered length when a record arrives in small pieces.

Affected: src/record.rs:154-210 (`next_message`, `materialize`, `find_resync_offset`).

Proof of Concept: Test A11 (release build):

```
R2 1600 x 40-byte records, one 64000-byte push: 1600 messages in 1.66 ms
R2 1600 x 40-byte records, pushed singly: 1600 messages in 143 us
R3 65543-byte record trickled 1 byte at a time: delivered 65535 B in 28.7 ms (about 2.1 GB copied)
R3 same record in 1460-byte chunks: delivered 65535 B in 51.5 us
R4 1000 x 64 KiB random pushes: 0 false headers, resyncs=1000, 153 ms, rss 3.7 MiB
R5 mutation fuzz 20 s: 10,302,511 records, 5,715,982 delivered, max rss 5.4 MiB, no panic
```

A peer that sends one byte per segment makes the client spend about 29 ms of copying per maximum-size record; against 64 KiB garbage the parser costs 0.15 ms per push and finds no false header in 64 MiB. No crash, no growth.

Impact: CPU amplification only, by a peer that already holds the carrier and could simply drop it. Not a vulnerability.

Preconditions: The server or an on-path attacker controlling segmentation.

Recommendation: Parse in place using `VecDeque::as_slices` or `make_contiguous`, keep a cursor, compact lazily, and remember the last scanned offset so that a short read does not rescan from the start.

Classification: CWE-407 · OWASP ASVS 5.0.0 V15 Secure Coding and Architecture · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/V

RVPN-01-509 WP5: run_tunnel trusts an unvalidated in-memory Config; mixed address families and an oversized prefix length panic (Low)

Fix Note: This issue has been fixed by the development team in commit cd99d0e (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d to be working as expected: `run_tunnel` calls `validate_public()` first (lib.rs:26) and `NetStack::new` returns `Result` for a family mismatch or an oversized prefix (netstack.rs:131-143). In B5 every unvalidated configuration that panicked before now returns a clean error (`must be the same address family`, `prefix_len 33 must be <= 32 for IPv4`, `mtu 0 out of range (576..=65503)`).

`Config::load` validates a file (config.rs:68-74), and `validate_public()` exists for configurations assembled in memory (config.rs:76-79), but `run_tunnel` never calls it (lib.rs:22-44). `NetStack::new` then reaches `panic!` for mixed families (netstack.rs:154) and smoltcp's `assert!(prefix_len <= 32)` for an IPv4 prefix above 32; an MTU of 0 or 70,000 is accepted silently and produces an unusable stack. The mobile embedders build the configuration in memory, which is exactly this path.

Affected: src/lib.rs:22-44 (`run_tunnel`); src/netstack.rs:130-163 (`NetStack::new`).

Proof of Concept: Test A9, each call in its own thread with the panic captured:

```
C6 run_tunnel(mixed families) : PANIC: address 10.50.0.2 and gateway fd00:5ed0:c0de::1 must be the same
    ↪ family
C6 run_tunnel(prefix_len=33) : PANIC: assertion failed: prefix_len <= 32 (vendor smoltcp wire/ipv4.rs
    ↪ :101)
C6 run_tunnel(mtu=0) : still running after 1.5 s (reconnect loop)
C6 run_tunnel(mtu=70000) : still running after 1.5 s (reconnect loop)
C6 run_tunnel(socks_listen=nonsense): returned Err: parse socks_listen nonsense
C6 run_tunnel(private key not base64): returned Err: private_key
```

Impact: In the CLI a panic terminates the process (fail-closed, since the SOCKS5 listener dies with it, lib.rs:65-75); in an embedder without `catch_unwind` the tunnel thread dies and the application's state machine may not notice. No network input reaches these paths.

Preconditions: An embedder-supplied misconfiguration.

Recommendation: Call `cfg.validate_public()` at the top of `run_tunnel`; make `NetStack::new` return `Result` instead of panicking; and reject MTU values outside 576 (1280 for IPv6) to 65,503 there as well.

Classification: CWE-248 · OWASP ASVS 5.0.0 V2 Validation and Business Logic · CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:N/VC:N/

RVPN-01-510 WP5: Public resolver addresses and an X25519 key pair embedded in unit tests contrary to the publication scrub (Info)

Fix Note: The key-pair part of this issue has been fixed by the development team in commit 7eab209 and verified by Red Team Ltd to be working as expected: in the updated working tree src/wg.rs is the only non-vendor file that changed, the previous X25519 pair is no longer present, lines 187-188 hold a different pair, and the derivation test passes against it (`cargo test --offline --locked derives_public_key_from_private`: 1 passed; full suite 84 passed; A22). The maintainers state that the replacement is the project's published example pair, the one shipped in the servers' config.example.toml; that statement could not be checked from this repository, whose own example files use placeholders, and the server repositories are outside WP5. The public resolver addresses remain by design and are accepted by the project as a documented risk (well-known public resolvers, no RostamVPN infrastructure); the recommendation below stands for them only as a hygiene option.

README.md:247-251 states that infrastructure addresses were removed for publication. The routable literal that the earlier static review found in a test fixture is gone (config.rs:150 and :174 now use an RFC 5737 address) and the example peer key in config.server-v6.example.toml:21 is now a placeholder. What remains are values outside the documentation ranges used as test destinations, and one real key pair used as a derivation vector. Red Team Ltd did not contact any of the addresses and lists them by class only.

Affected: src/carrier.rs:448 (public IPv4 resolver address); src/socks5.rs:343 (public IPv4 resolver address), :297, :346, :348 (public IPv6 resolver address); src/netstack.rs:387, :401 (public IPv6 resolver address), :422 (public IPv6 resolver address); src/server/addressing.rs:178 and src/server/engine.rs:380, :390 (public IPv6 resolver address); src/wg.rs:187-188 (a base64 X25519 private key and its public key used as a `wg pubkey` compatibility vector).

Proof of Concept: Static. A sweep of all non-vendor files for IPv4 and IPv6 literals outside loopback, RFC 1918, RFC 5737, RFC 3849 and fd00::/8, and for 44-character base64 strings, returned exactly the lines above (plus the all-A and all-B placeholder keys in bin/server.rs:254 and :266, which are not keys).

Impact: The addresses belong to well-known public resolvers, not to RostamVPN, so the exposure is nil; they are listed because the publication rule is categorical and because an automated scrub must not have to reason

about ownership. The key pair is almost certainly a throwaway generated for the test, but the tree gives no evidence of that, and a private key in a public repository is a value that must be provably throwaway.

Preconditions: None.

Recommendation: Replace the test destinations with RFC 5737 and RFC 3849 addresses; regenerate the derivation vector from a key whose provenance is recorded in a comment ("generated for this test on YYYY-MM-DD, never used elsewhere") or derive the expected public key from a fixed seed such as `[0x11; 32]` as the other tests do; add a pre-publication grep to CI.

Classification: CWE-540 · OWASP ASVS 5.0.0 V14 Data Protection · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:N/VA:N/S

RVPN-01-513 WP5: SOCKS5 implementation deviates from RFC 1928: method selected without being offered, success reported before the inner connection exists, refused connections appear as success then EOF, README IPv6 statement outdated (Low)

Fix Note: Partially fixed by the development team in commit 7fc5231 (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d: method selection follows RFC 1928 section 3 (socks5.rs:147-152; B9/S1 `05 ff` then close), a target of the other address family is refused at `NetStack::connect` and reported as 0x04 (netstack.rs:189-200; B9/D3 `05 04`), and README.md:228-230 now describes the single inner family correctly. What remains: the success reply is still sent before the inner connection exists, so a refused destination still appears as success followed by end-of-file (B9/D2: reply 0x00 after 0.5 ms, EOF after 0.7 ms).

RFC 1928 section 3 requires the server to select a method from those the client offered or to reply 0xFF; the handler replies 0x00 unconditionally (socks5.rs:72). Section 6 requires the reply to reflect the outcome of the CONNECT; the handler replies success as soon as the carrier has created a smoltcp socket and queued the SYN (socks5.rs:130-151, carrier.rs:217-248), before any inner handshake, so a refused or unreachable destination is reported as success followed by end-of-file instead of reply 0x05 or 0x04. README.md:228-229 states that IPv6 targets are refused with "ATYP not supported", but socks5.rs:105-111 parses and forwards them (verified by the crate's own test socks5.rs:282-313).

Affected: src/socks5.rs:61-72, :130-154 (`handle`); src/carrier.rs:217-248 (`open_stream`); README.md:228-229.

Proof of Concept: Test A13:

```
S1 greeting offering only method 0x02 -> reply: "05 00" (expected 05 ff)
S3 greeting with nmethods=0 -> reply: "05 00"
D2 CONNECT 10.50.0.1:9 (no inner listener, responder sends RST): reply code 0 after 635 us; then read ->
    ↪ 0 bytes (EOF) after 708 us
```

Impact: Clients cannot distinguish "connection refused" from "connected then closed", which breaks retry logic and error reporting in the applications; a client that insists on authentication is silently downgraded to none. Interoperability and correctness rather than security.

Preconditions: None.

Recommendation: Select only an offered method; defer the SOCKS5 reply until the smoltcp socket reaches Established (reply 0x00) or Closed (reply 0x05 for RST, 0x04 on timeout), which `pump_streams` can signal through the oneshot already used for the handles; correct README.md:228-229.

Classification: CWE-573 · OWASP ASVS 5.0.0 V2 Validation and Business Logic · CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:N/

RVPN-01-514 WP5: Configuration validation gaps: endpoint checked only for a colon, unknown keys silently ignored so a misspelled preshared_key disables the PSK, degenerate keys accepted, MTU above 65503 accepted though unsendable (Low)

Fix Note: This issue has been fixed by the development team in commit a2a8860 (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d to be working as expected: `Config` carries `#[serde(deny_unknown_fields)]` (config.rs:28), the endpoint must be `host:port` with a port in 1 to 65535 and a name, IPv4 literal or bracketed IPv6 literal as host (config.rs:191-213), all-zero and self keys are refused

(config.rs:131-139), and the MTU is capped at 65,503 (config.rs:11-13, :163-171). In B5 the misspelled pre-shared key, all seven malformed endpoints, the degenerate keys and `mtu = 65535` were rejected, and in B16 the CLI refused a configuration with an unknown key. Unknown configuration keys now fail at load with a TOML parse error that names the field.

`validate()` checks the endpoint only for the presence of a colon (config.rs:87-91); `Config` does not use `#[serde(deny_unknown_fields)]`, so a misspelled key is dropped without error; the all-zero private key and a peer key equal to the client's own key pass; and the MTU upper bound of 65,535 (config.rs:114-121) exceeds what the carrier can frame, because a full-size inner packet plus 32 bytes of WireGuard overhead plus the 8-byte header must not exceed 65,543 (record.rs:23, :42-44), so every inner packet above 65,503 bytes is dropped at `record::encode` with only a debug log (carrier.rs:204-206).

Affected: src/config.rs:23-61 (`Config`), :81-123 (`validate`); src/carrier.rs:201-207.

Proof of Concept: Test A9:

```
C2 TOML with misspelled preshared key parses: yes; validate_public(): Ok; resulting preshared_key: None
C3 validate_public(endpoint="::1"): Ok endpoint=":": Ok endpoint="a:b": Ok endpoint="203.0.113.20:99999":
    ↪ Ok
C5 validate_public(all-zero private key): Ok validate_public(peer key == own key): Ok validate_public(mtu
    ↪ =65535): Ok
C7 inner packet of 65504 B -> WG message 65536 B -> record::encode: Err("record length 65544 exceeds
    ↪ WG_MAX_PACKET_SIZE")
C6 run_tunnel(endpoint=::1): carrier connect failed: connect to ::1: failed to lookup address information
    ↪  ... (reconnect loop)
```

Impact: An operator who intends a pre-shared key and mistypes the field runs without it and is never told; an unparsable endpoint produces an endless reconnect loop with a misleading resolver error instead of a startup failure; an MTU above 65,503 yields a tunnel whose large packets vanish silently. None of these is reachable from the network.

Preconditions: A configuration error.

Recommendation: Add `#[serde(deny_unknown_fields)]` to `Config`; parse the endpoint as `SocketAddr` or as `host:port` with a port in 1 to 65,535; reject the all-zero private key, a peer key equal to the own public key and the all-zero peer key; cap `mtu` at 65,503; and promote the `record encode` failure in the carrier to a warning with a counter.

Classification: CWE-20 · OWASP ASVS 5.0.0 V13 Configuration · CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:N/VC:L/VI:N/VA:L/SC:N/

RVPN-01-515 WP5: WireGuard protocol deviations inherited from boringtun 0.7.1: unpadded data messages, no retransmission jitter, no message-count rekey limits, 1024-packet replay window (Info)

Fix Note: Partially fixed by the development team in commit c0fdb7d (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d: the client adds 0 to 333 ms of random delay to every timer tick (carrier.rs:198-200, :357-364), so handshake retries arrived 5.03 to 5.48 s apart (B8) and keepalives 25.02 to 25.44 s apart (B13, B14) instead of on fixed multiples of the tick. The jitter lives in the crate's loop, not in boringtun; padding of data messages, the message-count limits and the window size are unchanged.

The WireGuard protocol pads the plaintext of every data message to a multiple of 16 bytes, retries a handshake after REKEY_TIMEOUT plus a random jitter of up to 333 ms, rekeys after REKEY_AFTER_MESSAGES = 2^60 and rejects after REJECT_AFTER_MESSAGES = 2^64 - 2^13 - 1 messages, and the Linux reference implementation uses an anti-replay window several times larger than boringtun's (not verified in this tree). boringtun 0.7.1 as vendored does none of the first three: session.rs:211 carries the comment "spec requires padding to 16 bytes, but actually works fine without it", timers.rs:228-236 describes the jitter in a comment but implements a fixed REKEY_TIMEOUT, no reference to either message limit exists in the noise module, and the replay window is 1,024 packets (session.rs:36). The session index is deterministic for the reason given in RVPN-01-501. The library inherits all of this.

Affected: vendor/boringtun/src/noise/session.rs:36, :196-230; vendor/boringtun/src/noise/timers.rs:215-236; src/wg.rs:66-75 as the consumer.

Proof of Concept: Observed on the wire in tests A12, A13 and A18: data records of 80, 84 and 92 bytes (inner packets of 40, 44 and 52 bytes, not multiples of 16); initiation retries at 5.001, 5.252, 5.250, 5.249 s intervals, the 250 ms steps being the client's timer tick (carrier.rs:24), not jitter; the rekey at exactly 120.0 s after the session (the time-based limit works). Static: `grep -rn "REKEY_AFTER_MESSAGES\|REJECT_AFTER_MESSAGES" vendor/boringtun/src/noise/` returns nothing. The in-process replay tests (A10/E3) confirm the window at 1,024.

Impact: The missing padding exposes exact inner packet lengths (a traffic-analysis aid that matters less here than RVPN-01-501 but would remain after a wrapper is added); the missing jitter is a small fingerprint component; the message limits are unreachable in practice because the 120 s and 180 s time limits bound a session to far fewer than 2^60 messages; the smaller window is stricter, not weaker, and cannot matter over an in-order TCP carrier. Conformance rather than security.

Preconditions: None.

Recommendation: Track boringtun upstream for padding and jitter, or patch the vendored copy: pad `src` to a multiple of 16 in `format_packet_data`, add the jitter in `update_timers`, and enforce the message limits with a counter check in `encapsulate`. Record the deviation in the project's protocol documentation.

Classification: CWE-573 · OWASP ASVS 5.0.0 V11 Cryptography · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:N/VA:N/SC

RVPN-01-516 WP5: Userspace server runtime hardening gaps: per-datagram task and channel on the UDP listener, no carrier idle timeout or per-source limit, lock poisoning aborts, accept-error busy loop (Low)

Fix Note: Partially fixed by the development team in commit 1d39563 (maintainers' attribution) and verified by Red Team Ltd against the working tree at HEAD 27a912d: a TCP carrier that sends nothing for 540 s is dropped (run.rs:22-24, :117-120, test at :245-264), accept errors back off for 100 ms (run.rs:26-28, :69-72), and every peer and registry lock goes through a poison-tolerant helper (peers.rs:14-18, used throughout peers.rs and at engine.rs:127, :172, :186). What remains: the UDP listener still spawns a task and two channels per datagram (run.rs:166-174), `Engine` still allocates a 65,567-byte scratch per packet (engine.rs:126, :171, :184), and there is no per-source connection limit.

Static observations on the server runtime, none of which was exploited: the UDP listener spawns a task and allocates two channels for every datagram, including garbage (run.rs:152-160); a TCP carrier that never sends anything is kept for ever with a 64 KiB read buffer and a parser, with no idle timeout or per-source cap (run.rs:76-119); every `Mutex::lock().unwrap()` on the peer and registry state turns a poisoned lock into a process abort (engine.rs:128, :173, :187; peers.rs:55, :74, :115, :163-170, :177-181, :189, :194, :203, :209); `accept()` errors such as EMFILE are logged and retried immediately, which spins at full CPU once descriptors are exhausted (run.rs:57-64, mirrored on the client at socks5.rs:41-47); and `Engine::ingress` allocates a fresh 65,567-byte scratch vector per packet (engine.rs:126, :172, :185).

Affected: src/server/run.rs:57-64, :76-119, :142-171; src/server/engine.rs:126-128, :172-173, :185-187; src/server/peers.rs:55-74, :115-209; src/socks5.rs:41-47.

Proof of Concept: Static only; the code excerpt for the datagram path:

```
// run.rs:152–160 (per datagram)
let (peer_tx, mut peer_rx) = mpsc::channel::<Vec<u8>>(4);
let out = tx.clone();
tokio::spawn(async move { while let Some(m) = peer_rx.recv().await { ... } });
```

Impact: Cheaper flooding of the userspace server, a harder-to-diagnose failure mode under descriptor exhaustion, and an abort instead of a logged error if any peer operation ever panics while holding a lock. Low because the binary is not the production responder.

Preconditions: Network reach to the userspace server.

Recommendation: Keep one forwarding task per remote address with an expiry rather than one per datagram; add a carrier idle timeout (for example 3 x REJECT_AFTER_TIME) and per-source connection limits; replace `lock`

↪ `().unwrap()` with `lock().unwrap_or_else(|p| p.into_inner())` or `parking_lot::Mutex`; back off on `accept()` errors; reuse the scratch buffer.

Classification: CWE-400 · OWASP ASVS 5.0.0 V15 Secure Coding and Architecture · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/V

Conclusions

The redsocket_wgtcp client library is a small, legible piece of software that does one thing: it moves WireGuard messages over a TCP connection and terminates the inner TCP traffic in process. Red Team Ltd found the cryptographic and protocol core to be in good shape, both on paper and when driven with hostile input, and found the weaknesses where the library meets the hostile network: the carrier is identifiable from its first bytes, a single blocked write freezes the whole client, and the readiness signal the applications rely on is raised before anything has been proven. None of the sixteen findings allows a remote compromise of the client or a decryption of its traffic. Two of them, however, go to the heart of what a circumvention tool is for, which is why they are rated High despite leaving confidentiality intact. The sections below walk through each area examined, what held and what did not.

Handshake and key derivation. The library delegates the Noise IK handshake to boringtun 0.7.1 and gets the important properties for free: the server's static public key is pinned at `Tunn::new` (wg.rs:66-75), so an impersonator needs the server's private key; the optional pre-shared key is mixed into the chaining key on both sides (vendor handshake.rs:606-611); the initiation's static key is encrypted under the ephemeral-static DH so an observer learns nothing about the user's identity from the first message; and the responder enforces the TAI64N monotonicity check that stops replayed initiations (handshake.rs:542-547). Red Team Ltd exercised each of these: a replayed initiation was refused with WrongTai64nTimestamp, a replayed response with UnexpectedPacket, an initiation with a forged mac1 with InvalidMac, and the client correctly answered a cookie reply with a mac2-bearing retransmission exactly 5.002 s later (A10, A14, A15). Key generation uses the OS random source through `getrandom::fill` (lib.rs:85), and the crate's own test asserts distinctness and correct derivation. Two blemishes remain in this area, both in RVPN-01-515 and RVPN-01-501: boringtun does not pad data messages or jitter retransmissions, and the session index the library asks for is deterministic. The index is the one item the maintainers can fix in a single line.

Counter and replay window. boringtun's 1,024-bit sliding window behaved exactly as its code says (A10/E3): in-order packets accepted, a replay refused with DuplicateCounter, a late packet inside the window accepted, a packet 1,147 behind refused with InvalidCounter, a flipped ciphertext bit and trailing bytes refused with InvalidAeadTag, a foreign receiver index refused with NoCurrentSession, and the same results when the records travelled over the live carrier (A15). The window is smaller than the kernel implementation's 8,128 and the protocol's message-count limits are not enforced, but over an in-order TCP carrier the first cannot matter and the time-based limits make the second unreachable. The counter starts at zero in every session and increments by one, which is as specified; it is visible in clear and forms part of the fingerprint, but that is WireGuard's design and not the library's.

Carrier framing robustness. The record parser held up well. Sixteen hand-built headers with valid checksums were accepted or rejected exactly at the documented bounds (40 to 65,543 bytes, type 0, flags 0 or 1), 64 MiB of random bytes produced no false header, a 20-second mutation fuzz of ten million records produced no panic and a resident set that never exceeded 5.4 MiB, and over the live carrier every replayed, flipped, oversized, misframed and fragment-flagged record was rejected while the tunnel kept echoing data (A11, A15). The parser is memory-bounded by construction. Its only weakness is efficiency: it copies and rescans its whole buffer on every call, which an adversarial peer can turn into 29 ms of copying per maximum-size record (RVPN-01-508). The framing itself is the problem, not the parser: an 8-byte plaintext header with a public checksum, no padding and no cover makes every connection identifiable from its first segment (RVPN-01-501). Red Team Ltd captured the same sixteen leading bytes on all eight connections observed, and the session index in bytes 12 to 15 counted 1, 2, 3 across retransmissions. This is the finding that decides whether the transport can be used in a censored region at all, and it is shared with the kernel module (WP6), which speaks the same bytes.

Timers, rekey and keepalive. The time-based protocol limits are implemented correctly and were observed on the wire: persistent keepalives at 25.00 s intervals, a rekey initiation at 120.0 s after the session when data had been sent, the handshake retransmission every 5 s, and the 90 s expiry of a handshake that is never answered

(A12, A17, A18); the 180 s REJECT_AFTER_TIME expiry was not reached in any run because the rekey pre-empts it. The client drives all of this from a 250 ms tick (carrier.rs:24), and the 0.25 s quantisation is visible in the retry timestamps. The cookie path works. What does not hold is the coupling of the timers to the socket: because `update_timers` runs in the same loop that writes to the carrier, a peer that stops reading silences the watchdog entirely (RVPN-01-511). In test A16 the client sat for 120 s with its send queue full, the flag reading up, a new CONNECT never answered, and not one "connection expired" line in its log; the same client against a merely silent peer gave up at 90.01 s. The fix is architectural but small: a dedicated writer task with a bounded channel and a deadline, which the in-tree server already has.

Endpoint and roaming authentication. On the client side the endpoint is simply what the configuration names; the Noise handshake is the only authentication and it is sufficient. On the in-tree server, Red Team Ltd looked for the classic roaming mistake of adopting a new transport address on an unauthenticated packet and did not find it: a carrier becomes a peer's path only after a data packet has decrypted under the peer's session and its inner source address has matched the peer's allowed IP (engine.rs:137-148, run.rs:188-194), and a delayed packet with an older generation cannot drag the peer back (peers.rs:56-59). The crate's own tests cover both properties and passed. The anti-spoofing check applies to both address families and the stateless IPv6 derivation uses a domain-separated BLAKE2s over the public key with every key byte affecting the output (addressing.rs:11-45, tests at :144-157). This area held.

Configuration parsing and secret handling. The TOML loader is robust against hostile input: a 10 MiB value, 274 truncations, 100,000 nested brackets and every mistyped field produced a clean error without a panic (A9). The validation logic is where the gaps are: the endpoint is checked only for a colon, unknown keys are ignored so that a misspelled `preshared_key` silently disables the PSK, degenerate keys pass, and the MTU bound exceeds what the carrier can frame (RVPN-01-514). `run_tunnel` does not validate at all, so an embedder that assembles a configuration in memory can panic the library on a family mismatch or a bad prefix length (RVPN-01-509). Secrets are handled carelessly rather than dangerously: the private key is a plain `String` in a struct that derives `Debug` and `Serialize`, so any formatting of the configuration prints it, and none of the crate's copies is zeroised (RVPN-01-506). The vendored x25519-dalek zeroises boringtun's copy on drop, and the CLI never logs the private key (A19), which is why the item is Low rather than Medium.

Error handling and half-open tunnels. Apart from the stall, the carrier fails closed and recovers quickly: a peer-initiated close in the middle of a record produced "carrier lost: carrier closed by peer", a reconnect after the 500 ms minimum back-off and a working tunnel 1.0 s later; a reset of the stalled carrier recovered in 1.5 s; the back-off doubles to a 30 s ceiling (carrier.rs:31-32, :114-116); and there is no direct-connect fallback anywhere, so no application byte can leave the device outside the tunnel. Two behaviours deserve the maintainers' attention. The readiness flag is set on TCP connect rather than on handshake completion (RVPN-01-502), which turns every accept-and-discard middlebox into a "connected" display for 90 s at a time. And open streams survive a carrier loss untouched: `Carrier::run` resets only the parser (carrier.rs:112), the smoltcp sockets are not polled while the carrier is down, so their 30 s timeout does not run either, and the application sees its connection hang until the carrier returns and the sockets time out. That is a defensible design for a transparent reconnect, but it should be a documented decision rather than an accident of the loop structure.

Resource bounds. Everything that can grow is bounded: the carrier read buffer, the boringtun scratch buffer, the per-stream buffers and channels, the connect queue, boringtun's packet queue and the parser's maximum record. Red Team Ltd measured rather than assumed: 500 idle SOCKS5 connections cost 0.6 MiB, 200 held CONNECTs cost 4.6 MiB (23 KiB per stream, because the 512 KiB of zero-filled buffers are not resident until used), the garbage flood and the fuzz stayed under 6 MiB (A12, A11). The earlier static review's figure of half a megabyte per stream did not survive measurement. What is missing is a cap on the number of streams and connections and a timeout on an idle SOCKS5 request (RVPN-01-504); the first resource to run out is file descriptors, not memory.

SOCKS5 front-end. The listener parses its protocol safely (SOCKS4, zero methods, zero-length domain, truncated and unknown address types all ended in a clean reply or a closed connection) but it is unauthenticated and selects method 0x00 whatever the client offered, which on a device where every application can reach loopback means every application can use, detect and tie up the tunnel (RVPN-01-504, also WP3). It also reports success before the inner connection exists, so a refused destination looks like a connection that closed immediately (RVPN-01-513), and it resolves host names with the host resolver outside the tunnel (RVPN-01-503). The README is honest about the last point but then documents `--socks5-hostname` as the way to use the

client, and it still says IPv6 targets are refused although the code forwards them.

Unsafe code and FFI surface. There is no `unsafe` in src/ or examples/. The library exposes no C ABI; the Android C ABI and JNI live in the sibling wgtcp-android crate and are WP3's concern. The socket-protection hook is a plain `fn(i32)` installed once and refused a second time (carrier.rs:338-343, test at :405-433), invoked on a blocking thread before `connect` (carrier.rs:356-371); with the hook installed only the first resolved address is dialled, which loses the address-family fallback that `TcpStream::connect` would otherwise provide. The unsafe surface the client inherits is in the vendored crates (Appendix B): ring and curve25519-dalek for the arithmetic, tokio, mio, socket2, libc and nix for the system interface, and, only because it is not feature-gated, the tun crate with its Windows bindings.

Dependency hygiene and vendoring. The supply-chain posture is better than most: every external package comes from crates.io with a checksum, there are no git or path sources, all 171 crates are vendored and the build is forced onto them, build paths are remapped so the binary does not reveal the developer's home directory, and cargo audit reports no vulnerable crate. The remaining items are the yanked chacha20 0.10.1 reached only through a redundant `rand` dependency, duplicated crate versions, an over-featured tokio, the tun crate in the library's graph, the absent toolchain pin and the absent release profile (RVPN-01-507). There is no SBOM; the lockfile serves as one, and generating CycloneDX 1.7 or SPDX 3.0.1 from it is a one-command addition that the EU Cyber Resilience Act's documentation duties will make useful.

The userspace server. The server is not the production responder, but it is in the tree and the mobile applications vendor the crate that contains it. Its peer logic is careful (anonymous initiator identification without cleartext identity, anti-spoofing on both families, self-as-peer refusal, monotonic carrier adoption), and its write path is the model the client should adopt. Its handshake path is not: every 148-byte message that begins with the type byte 1 costs an X25519 operation before any authentication, a hundred times the cost of any other garbage, and the cookie mechanism that WireGuard provides against exactly this load can never engage because the engine passes no source address to the rate limiter (RVPN-01-512). A handful of runtime hardening gaps complete the picture (RVPN-01-516).

Documentation and publication hygiene. The README is unusually candid: it states the DNS limitation, the TCP-over-TCP risk, the lack of inner UDP and the scepticism due to its own benchmark figures. The publication scrub was effective: the routable literal that the earlier review found is gone, the example server key is a placeholder, licence headers are on every file and the GPL oracle is absent. What remains are public resolver addresses used as test destinations and one key pair used as a derivation vector (RVPN-01-510), listed here by class so that an automated scrub can remove them without judgement.

Remediation (version 1.1). Between the two issues of this report the maintainers landed fifteen commits, and Red Team Ltd re-ran the harness against the result (Appendix A, B1 to B18). The readiness flag now follows the handshake and is cleared by a drop guard, which closes RVPN-01-502 completely: against the discard listener it never rose in 100 s, and a shutdown cleared it within 22 ms. The configuration class is closed: unknown keys, malformed endpoints, degenerate keys and an unframeable MTU are refused, `run_tunnel` validates first and the stack no longer panics (509, 514). Logging is down to key prefixes with no destinations (505), the parser is linear (508), the yanked crate is gone and the server is behind a feature so that embedders no longer compile the TUN crate (507 in part). The stall is bounded rather than removed: a peer that stops reading is dropped after 50 s and the tunnel recovers, but the loop still pauses for those 50 s (511). The carrier fingerprint lost three of its sixteen constant bytes and gained jitter, nothing more (501, 515). The SOCKS5 front now follows RFC 1928 for method selection, times out idle negotiations and caps streams, but remains unauthenticated and still reports success before the inner connection exists (504, 513). The server gained an idle timeout, accept back-off and poison-tolerant locks, but still pays the Diffie-Hellman before mac1 and still cannot issue cookies (512 open, 516 in part). Host names still leave the device in clear (503 open).

Verdict. As a WireGuard implementation the library is correct and robust: Red Team Ltd could not forge, replay, reorder, truncate or corrupt its way past the Noise handshake or the transport, could not crash the parser or the loader with any input, and found the time-based limits implemented to the second. As a circumvention transport the library is not yet fit for a hostile network: the carrier announces itself in its first sixteen bytes, a keyless middlebox can freeze it indefinitely while it reports itself up, and the readiness signal the applications display is raised before the server has said a word. These were design-level items with small code footprints, and the remediation pass has dealt with the readiness signal outright and bounded the stall; what still stands

between the library and a hostile network is unchanged in kind: the carrier announces itself in its first thirteen bytes, host names leave the device in clear, the loopback proxy is open to every local process, a stalling peer freezes the client for up to 50 s per occurrence, and the userspace server can be made to spend a Diffie-Hellman on every forged initiation. Those are the open work; the rest is hygiene that the pass has largely absorbed.

Prioritised recommendations.

1. Treat RVPN-01-501 as a release blocker for censored regions: wrap the record stream in a cover protocol shared with the kernel module, pad records and draw a fresh index per handshake rather than per process (open after the pass).
2. Finish RVPN-01-511: move writes to a dedicated task fed by a bounded channel and keep the timers on their own task, so that the 50 s deadline bounds a background write rather than the whole loop; set `TCP_USER_TIMEOUT` where available.
3. Fail loudly on host-name targets until in-tunnel resolution exists, and require an IP-literal endpoint (RVPN-01-503, open).
4. Authenticate the SOCKS5 front with a per-launch RFC 1929 secret, refuse non-loopback binds by default and allocate buffers lazily (RVPN-01-504 residual); defer the SOCKS5 reply until the inner connection is established or refused (RVPN-01-513 residual).
5. On the server, verify mac1 before the Diffie-Hellman and pass the source address so cookies engage (RVPN-01-512, open); replace the per-datagram task and add a per-source cap (RVPN-01-516 residual).
6. Wrap secrets in zeroising types, drop `Serialize` from the secret fields and refuse a group- or world-readable configuration (RVPN-01-506 residual).
7. Pin the toolchain, complete the release profile, publish an SBOM and add cargo audit and cargo deny to CI (RVPN-01-507 residual).
8. Track boringtun for data-message padding and the message-count limits, or patch the vendored copy (RVPN-01-515 residual), and export a three-state readiness value with the handshake age so that embedders can show “reconnecting” during the bounded pause (RVPN-01-502 follow-up).

Red Team Ltd thanks the RostamVPN maintainers for a clean, well-tested and candidly documented code base, which made both the review and the dynamic work straightforward, for the earlier static review whose hypotheses gave this assessment a precise starting point, and for a remediation pass that arrived within a day, came with tests for each change and left the harness able to confirm every claim.

Appendix A: Dynamic test log

Environment for every entry: macOS 25.6.0 (Darwin) on Apple Silicon; rustc 1.91.1 (2025-11-07), cargo 1.91.1, clippy 0.1.91, cargo-audit 0.22.2 with RustSec advisory database commit edb58f59 of 2026-10-14 (1,329 advisories); Python 3.15.0; pandoc 3.8.2.1. Working directory /tmp/rvpn01/wp5/repo (a symbolic link to the repository checkout at commit 05b491a) unless stated; `CARGO_TARGET_DIR=/tmp/rvpn01/wp5/target` exported for every cargo command against the repository. Scratch crate: /tmp/rvpn01/wp5/harness (wp5harness, release build, depends on the library by path, builds from the repository’s vendored sources). Log files under /tmp/rvpn01/wp5/logs. Timestamps in the logs are UTC (2026-10-15T02:10Z to 02:26Z, that is 2026-10-14 19:10 to 19:26 local time). Every process started is listed with its PID; all harness processes self-terminated at the end of their scenario, the CLI process in A19 was stopped by PID, and `ps` and `lsof` showed no remaining process or bound port in 20500 to 20599 afterwards.

A1 Build. `cargo build --offline --locked` (shell PID 64861, 02:10:03Z). Result: pass. Output (trimmed): `Compiling boringtun v0.7.1 ... Compiling redsocket-wgtcp v0.1.0 ... Finished dev profile [unoptimized + debuginfo] target(s) in 6.20s`; 128 crates compiled; binaries redsocket-wgtcp (16.3 MB) and redsocket-wgtcp-server (15.6 MB) and libredsocket_wgtcp.rlib produced.

A2 Unit tests. `cargo test --offline --locked`. Result: pass. Output: `running 82 tests ... test result: ok. 82 passed; 0 failed; 0 ignored` (library), `running 0 tests` (bin redsocket-wgtcp), `running 2 tests ... test result: ok. 2 passed` (bin redsocket-wgtcp-server), `running 0 tests` (doc tests). Note: the crate’s tests bind 127.0.0.1:0 (ephemeral ports chosen by the target’s test code).

A3 Lints. `cargo clippy --offline --locked --all-targets`. Result: pass, `Finished dev profile ... in 3.26s`, no warnings emitted.

A4 Examples. `cargo build --offline --locked --examples` then `cargo run --offline --locked --example genkey` ↪ . Result: pass; two lines of 44 characters each ending in "=" (a base64 private and public key; values not recorded).

A5 CLI surface. `cargo run --offline --locked --bin redsocket-wgtcp -- genkey` (lines `private:` and `public:` with 44-character values, not recorded); `-- --help` (subcommands run, genkey, pubkey); `cargo run --offline --` ↪ `locked --bin redsocket-wgtcp-server -- --help` (`--config <CONFIG>`). Result: pass.

A6 Dependency audit. `cargo audit --no-fetch`. Result: 0 vulnerabilities, 1 warning. Output: `Loaded` ↪ `1329 security advisories ... Scanning Cargo.lock for vulnerabilities (172 crate dependencies)/ Crate:` ↪ `chacha20 Version: 0.10.1 Warning: yanked / warning: 1 allowed warning found.`

A7 Duplicate crates. `cargo tree --offline --locked -d` (host target). Result: base64 (0.22.1 via boringtun, 0.23.1 direct), bitflags (1.3.2 via smoltcp, 2.13.1 via nix), chacha20 (0.9.1 via chacha20poly1305, 0.10.1 via rand), getrandom (0.2.17 via rand_core 0.6 and ring, 0.4.3 direct and via rand) and rand_core (0.6.4, 0.10.1) in two versions each, and memchr listed twice under one version (normal and build dependency). The vendor directory additionally holds two versions each of cpufeatures, defmt, syn and windows-sys, which are resolved only on other targets or features: nine crates in two versions in total.

A8 Harness build. In /tmp/rvpn01/wp5/harness: `cargo build --offline --release`. Result: pass after one type fix; binary target/release/wp5harness (4.9 MB).

A9 Configuration loader (config-fuzz). `RUST_LOG=debug ./harness/target/release/wp5harness config-fuzz` (PID 95072). Result: 2 panics on unvalidated input, 3 validation gaps, no panic in the TOML layer. Trimmed output:

```
C1 Config Debug output contains the private key verbatim: true
C1 toml::to_string(&Config) re-emits the private key: true
C2 TOML with misspelled preshared key parses: yes; validate_public(): Ok; resulting preshared_key: None
C3 validate_public(endpoint="::1"|":"|"a:b
    ↪ "|"203.0.113.20:99999"|"203.0.113.20:0"|"[::1]"|"203.0.113.20:51820:x"): Ok (all)
C4 10 MiB private_key: parse true in 8.3 ms; validate Err("private_key") in 2.3 ms
C4 truncated TOML prefixes (ok/err): 11/274; 100k nested brackets: Err without panic; mistyped fields:
    ↪ Err (all)
C5 validate_public(prefix_len=33|255, mtu=0|100|70000, mixed families, empty key, psk ""): Err; (mtu
    ↪ =65535, keepalive=0, all-zero key, peer key == own key): Ok
C6 run_tunnel(mixed families): PANIC netstack.rs:154; run_tunnel(prefix_len=33): PANIC smoltcp assertion
    ↪ prefix_len <= 32
C6 run_tunnel(mtu=0|70000|endpoint=::1): reconnect loop; (socks_listen=nonsense|bad key): clean Err
C7 inner 65503 B -> encode Ok; 65504 B -> Err(record length 65544 exceeds WG_MAX_PACKET_SIZE)
```

A10 WireGuard engine (engine-cost). `./harness/target/release/wp5harness engine-cost` (PID 95330). Result: 100x cost asymmetry on the server engine; all replay and integrity checks behave as specified. Trimmed output:

```
E1 ingress(valid-mac1 initiation, unregistered key): 32.2 us/msg; (random type=1): 31.7 us/msg; (random
    ↪ type=9): 0.3 us/msg; (random type=4): 0.2 us/msg
E2 initiation 148 B type 1 sender_idx 1; mac1 valid; mac2 zero; response 92 B type 2
E2 server: replayed initiation -> Err(WrongTai64nTimestamp); client: replayed response -> Err(
    ↪ UnexpectedPacket)
E3 counter 0 ok; 0 replayed -> DuplicateCounter; 2 before 1 ok; 1 late ok; 1 replayed -> DuplicateCounter
E3 1150 ok; 3 (1147 behind) -> InvalidCounter; 1000 (150 behind) ok; flipped bit -> InvalidAeadTag;
    ↪ intact after -> ok
E3 foreign index -> NoCurrentSession; 20-byte message -> InvalidPacket; 100 trailing bytes ->
    ↪ InvalidAeadTag
E4 initiation with random mac1 -> InvalidMac; 50 valid-mac1 initiations in 1 s: 8 processed, 42 UnderLoad
E5 encapsulate() before handshake -> initiation (type 1, 148 B); further packets queued
```

A11 Record parser (record-fuzz). `./harness/target/release/wp5harness record-fuzz` (PID 95355). Result: no panic, memory bounded, quadratic trickle cost. Output: see RVPN-01-508 (R1 sixteen headers at the documented bounds; R2 1.66 ms batched vs 143 us singly; R3 28.7 ms trickled vs 51.5 us chunked; R4 1000 x 64 KiB garbage, 0 false headers, 153 ms, rss 3.7 MiB; R5 10,302,511 records in 20 s, max rss 5.4 MiB).

A12 Accept-and-discard listener. `RUST_LOG=debug ./harness/target/release/wp5harness discard --port 20500` ↪ `--socks 20501 --secs 100` (PID 2841). Result: readiness flag true without a handshake; expiry at 90.01 s; retry cadence 5.0 to 5.25 s; indices 1 to 18 on the first connection, 19 on the second; SOCKS battery S1 to S7 as

quoted in RVPN-01-504; 200 CONNECTs +4.6 MiB. First bytes: `00 00 00 9c 00 00 b6 25 01 00 00 00 01 00 00 00`.

A13 Tunnel baseline and SOCKS battery. `RUST_LOG=debug ./harness/target/release/wp5harness tunnel --mode baseline --port 20502 --socks 20503 --secs 60` (PID 2844). Result: handshake at t=0.204 s, echo round trip at 0.516 s, 8 MiB echoed at 1,114 Mbit/s with integrity; server first bytes `00 00 00 64 00 00 a9 25 02 00 00 00 01 01 00 00`; S1 "05 00" for an unoffered method; S2 SOCKS4 reset; S3 "05 00" for zero methods; S4 reply 0x04 for a zero-length domain; S5 no request timeout; S6 500 idle connections +0.7 MiB, fresh CONNECT served; D1 "localhost" resolved by the host to 127.0.0.1 and tunnelled; D2 closed inner port: reply 0x00 after 635 us then EOF after 708 us.

A14 Tunnel cookie. ... `tunnel --mode cookie --port 20504 --socks 20505 --secs 40` (PID 2847). Result: initiation at 0.204 s withheld and answered with a 72-byte cookie reply; retransmitted initiation at 5.206 s carried a valid mac2 for the cookie (mac1 valid); handshake completed at 5.206 s; echo at 5.216 s. Flag true at 0.456 s.

A15 Tunnel replay and malformed records. ... `tunnel --mode replay --port 20506 --socks 20507 --secs 90` (PID 2850). Result: I1 replayed response -> `decapsulate: UnexpectedPacket`; I2 replayed data record -> `DuplicateCounter`; I3 flipped bit -> `DuplicateCounter` (counter check precedes decryption); I4 65,543-byte record with random body -> `InvalidPacket`; I5 1 MiB garbage -> resync, no error; I6 FRAG_FLAG record wrapping a fresh keepalive -> no error; I7 header claiming 200 B followed by 50 B -> stream held until close; after every case a 256 KiB echo succeeded with integrity and the resident set stayed between 10.7 and 16.1 MiB; I8 responder close mid-record at 6.272 s -> `carrier lost: carrier closed by peer`, reconnect, echo at 7.287 s (second connection index 2).

A16 Tunnel zero-window stall. ... `tunnel --mode stall --port 20508 --socks 20509 --secs 200` (PID 2853), responder receive buffer 32 KiB, RST on close. Result: responder stopped reading at 8.517 s with 3.66 GB already drained; client Send-Q 654,892 and responder Recv-Q 430,560 constant from +5 s to +120 s; `carrier_is_up()` true at +5, +15, +30, +60, +90, +120 s; new CONNECT at +15 s: no reply within 30 s; no "carrier lost" or "connection expired" in the client log during the stall; RST at 128.6 s -> `carrier write: Broken pipe`, then `connection expired` on the next tick, tunnel recovered after 1.52 s.

A17 Tunnel idle. ... `tunnel --mode idle --port 20510 --socks 20511 --secs 215` (PID 2856). Result: initiation 0.203 s, response 0.204 s, keepalive 0.204 s; client keepalives (40-byte records) at 25.21, 50.21, 75.29, 100.29 s; rekey initiation at 120.29 s with response and keepalive; keepalives at 125.29, 150.29, 175.54, 200.56 s; intervals 25.00, 25.01, 25.08, 25.00, 20.00, 5.00, 25.00, 25.25, 25.02 s.

A18 Tunnel rekey. ... `tunnel --mode rekey --port 20512 --socks 20513 --secs 140` (PID 2860), 1 KiB written every 5 s. Result: session at 0.204 s; rekey initiation at 120.219 s (response 120.219 s); 107 records total; keepalive intervals 25.00, 25.00, 25.01, 25.00 s before the rekey.

A19 CLI run. In /tmp/rvpn01/wp5/cli with a configuration written at umask 022 (mode -rw-r--r--, throwaway keys, endpoint 127.0.0.1:20599 with nothing listening, socks_listen 127.0.0.1:20520): `redsocket-wgtcp run --config config.toml` (PID 10649, stopped after 3 s by `kill 10649`); `printf '%s\n' "$SK" | redsocket-wgtcp pubkey` (44-character output). Result: configuration loaded without a permission warning; stderr at the default level contained the client public key, the full peer public key (1 match) and the endpoint; the private key did not appear (0 matches); connect failures logged with the back-off 500 ms, 1 s, 2 s. The configuration file was deleted afterwards.

A20 Tree integrity. `find . -type f ! -path './vendor/*' ! -path './target/*' ! -path './.git/*' -exec shasum -a 256 {} + | sort -k2` before any cargo command (31 files) and again after all tests. Result: identical; no target/ directory was created in the repository.

A21 Render check. `pandoc PENTEST-REPORT.md -t html -o /tmp/rvpn01/wp5/render.html`. Result: parses without error.

A22 Fix verification for RVPN-01-510 (after the assessment campaign). With the working tree updated to the maintainers' commit 7eab209: `cargo test --offline --locked derives_public_key_from_private` (05:02:44Z on 2026-10-15, that is 22:02 local on 2026-10-14). Result: `test wg::tests::derives_public_key_from_private ... ok`, 1 passed; the full suite `cargo test --offline --locked` again reported 82 + 2 passed, 0 failed. A checksum comparison against the 05b491a snapshot showed src/wg.rs as the only changed non-vendor file

(270 lines before and after), a sweep for 44-character base64 strings found exactly two in src/wg.rs (lines 187 and 188, distinct from each other) and the previous pair absent. This run is not counted among the twenty-one runs of the campaign.

Re-verification of the remediation pass (version 1.1)

Environment as above; working tree at HEAD 27a912d (maintainers' commits 22dba80 to 27a912d on top of 7eab209); runs on 2026-10-15 between 06:39Z and 06:46Z (2026-10-14 23:39 to 23:46 local). Thirteen harness and CLI process starts (the eleven scenarios and the two CLI invocations below) plus the cargo invocations of B1 to B4; every harness process self-terminated, the first CLI process was stopped by PID and the second exited on its own; `ps` and `lsof` showed no remaining process or bound port afterwards. These runs are not counted among the twenty-one of the original campaign.

B1 Build, tests, lints, embedder build, release strip. `cargo build --offline --locked` (pass); `cargo build --offline --locked --no-default-features --lib` (pass: the library builds without the server feature and the TUN crate); `cargo test --offline --locked` (`running 100 tests ... 100 passed; 0 failed`, bin `redsocket-wgtcp-server 2 passed`, 0 doc tests); `cargo clippy --offline --locked --all-targets` (no warnings); `cargo build --offline --locked --release` (release binary 3,141,248 bytes, 114 symbols reported by `nm`, against 16,347,808 bytes with symbols before); `cargo run --offline --locked --example genkey` (two 44-character lines).

B2 Dependency audit. `cargo audit --no-fetch`: `Loaded 1329 security advisories ... Scanning Cargo.lock for vulnerabilities (166 crate dependencies)`, no vulnerabilities, no warnings.

B3 Duplicate crates. `cargo tree --offline --locked -d` (host target): base64 0.22.1 and 0.23.1, bitflags 1.3.2 and 2.13.1, getrandom 0.2.17 and 0.4.3, syn 2.0.119 and 3.0.3, plus memchr and serde_core each listed twice under one version; the vendor directory additionally holds defmt and windows-sys in two versions for other targets. Vendor directories removed since 05b491a: chacha20 0.10.1, cpufeatures (second copy), errno, rand, rand_core 0.10.1, signal-hook-registry; 165 directories remain.

B4 Harness rebuild. In /tmp/rvpn01/wp5/harness, after changing the tokio feature list to the crate's six features and `NetStack::new(...)` to `NetStack::new(...).expect(...)` in the responder: `cargo build --offline --release` (pass). Scenario extensions for this version: a `shutdown` mode, a stream-cap check (S8), a family-mismatch check (D3), initiation-interval and ever-up summaries in the discard scenario, and a revised stall timeline.

B5 Configuration loader (config-fuzz). `RUST_LOG=debug ./harness/target/release/wp5harness config-fuzz` (PID 73985). Result: no panic anywhere. Trimmed output:

```
C1 Config Debug output contains the private key verbatim: false
C1 toml::to_string(&Config) re-emits the private key: true
C2 TOML with misspelled preshared key parses: no: TOML parse error ... unknown field `preshared-key`,
    ↪ expected one of ...
C3 validate_public(endpoint="::1"|":"|"a:b
    ↪ "|"203.0.113.20:99999"|"203.0.113.20:0"|"[::1]"|"203.0.113.20:51820:x"): Err (all);
    ↪ "127.0.0.1:20599": Ok
C5 validate_public(mtu=65535): Err("mtu 65535 out of range (576..=65503)"); (all-zero private key): Err;
    ↪ (peer key == own key): Err
C6 run_tunnel(mixed families): returned Err: ... must be the same address family
C6 run_tunnel(prefix_len=33 (v4)): returned Err: prefix_len 33 must be <= 32 for IPv4
C6 run_tunnel(mtu=0|70000): returned Err: mtu ... out of range (576..=65503); (endpoint=::1): returned
    ↪ Err: endpoint host ":" must be ...
```

B6 WireGuard engine (engine-cost). `./harness/target/release/wp5harness engine-cost` (PID 74003). Result: unchanged class behaviour: 23.9 us per valid-mac1 initiation from an unregistered key and 23.6 us per random type-1 message against 0.3 us and 0.2 us for other garbage (RVPN-01-512 open); initiation sender index now random (741932545 in this run); replay, window and integrity outcomes identical to A10.

B7 Record parser (record-fuzz). `./harness/target/release/wp5harness record-fuzz` (PID 74017). Result: R2 1,600 records in one push 40.8 us (A11: 1.66 ms), pushed singly 62.5 us; R3 maximum record trickled byte by byte 303.5 us (A11: 28.7 ms), in 1460-byte chunks 12.4 us; R4 1000 x 64 KiB garbage 142.6 ms, 0 false headers, rss 3.0 MiB; R5 12,718,106 records in 20 s, 7,054,972 delivered, max rss 4.7 MiB, no panic.

B8 Accept-and-discard listener. `RUST_LOG=debug ./harness/target/release/wp5harness discard --port 20500 --socks 20501 --secs 100` (PID 82007). Result: `carrier_is_up()was ever true during this scenario: false`; first bytes `00 00 00 9c 00 00 b6 25 01 00 00 00 01 78 ac 44`, second carrier **...** `13 78 ac 44` (counter 19, same random upper bytes); expiry at 90.11 s; initiation retry intervals 5.407, 5.388, 5.310, 5.356, 5.036, 5.386, 5.415, 5.258, 5.118, 5.314, 5.189, 5.155, 5.464, 5.044, 5.475, 5.242, 5.025 s; S1 `05 ff` then EOF; S3 `05 ff`; S5 closed after 10.0 s; S6 500 idle connections +0.8 MiB, fresh CONNECT served; S7 200 CONNECTs +4.7 MiB (24 KiB per stream); S8 cap engaged at the 512th concurrent stream with reply 0x01 (511 held by the harness plus one still in teardown).

B9 Tunnel baseline and SOCKS battery. **...** `tunnel --mode baseline --port 20502 --socks 20503 --secs 60` (PID 82010). Result: `wireguard handshake completed` logged at 0.203 s and the flag rose only after it; echo 8 MiB at 1,317 Mbit/s with integrity; server first bytes unchanged (`00 00 00 64 00 00 a9 25 02 00 00 00 01 01 00 00`); S1 `05 ff` then EOF; S2 reset; S3 `05 ff`; S4 reply 0x04; S5 closed after 10.0 s; S6 500 idle +0.7 MiB; D1 "localhost" still resolved by the host and tunnelled to 127.0.0.1:9; D3 IPv6 literal on the IPv4 tunnel: reply `05 04`; D2 closed inner port: reply 0x00 after 0.5 ms then EOF after 0.7 ms (unchanged); no `tunneling to` line at debug level.

B10 Tunnel cookie. **...** `tunnel --mode cookie --port 20504 --socks 20505 --secs 40` (PID 82013). Result: initiation withheld at 0.202 s (random sender index), cookie reply sent, retransmission at 5.703 s with a valid mac2, handshake and echo at 5.714 s.

B11 Tunnel replay. **...** `tunnel --mode replay --port 20506 --socks 20507 --secs 90` (PID 82020). Result: I1 to I7 rejected with the same errors as A15 (UnexpectedPacket, DuplicateCounter x2, InvalidPacket x2, resync, no error for the fragment record); the replayed response did not raise the readiness flag a second time; I8 close mid-record: `carrier lost: carrier closed by peer`, flag false at 6.996 s, reconnect, `wireguard handshake completed`, flag true at 7.499 s, echo at 7.954 s.

B12 Tunnel zero-window stall. **...** `tunnel --mode stall --port 20508 --socks 20509 --secs 200` (PID 82023), responder receive buffer 32 KiB. Result: responder stopped reading at t = 8.514 s with 3.97 GB drained; flag true and queues constant at +5, +15, +30, +45 s; `carrier lost: carrier write: peer stopped reading for 50s` at t = 58.49 s (stall + 49.98 s); flag false at 58.56 s; `carrier established` at 58.99 s; the CONNECT issued at stall + 15 s received reply 0x00 after 35.5 s; after the responder released its stalled loop (harness artefact, t = 60.63 s) the new carrier completed its handshake and the flag was true at 60.82 s; echo round trip at 98.63 s.

B13 Tunnel idle. **...** `tunnel --mode idle --port 20510 --socks 20511 --secs 110` (PID 82026). Result: keepalives at 25.55, 50.68, 75.70, 100.82 s; intervals 25.35, 25.13, 25.02, 25.12 s; no rekey within 110 s.

B14 Tunnel rekey. **...** `tunnel --mode rekey --port 20512 --socks 20513 --secs 140` (PID 82029). Result: rekey initiation at 120.219 s after a session at 0.203 s; keepalive intervals 25.44, 25.24, 25.16, 25.11 s before the rekey.

B15 Tunnel shutdown. **...** `tunnel --mode shutdown --port 20514 --socks 20515 --secs 30` (PID 82038). Result: flag true before shutdown; `tunnel shutting down on request, run_tunnel returned: Ok(()); carrier_is_up() cleared after shutdown: yes, within 21.8ms.`

B16 CLI run. In /tmp/rvpn01/wp5/cli2 with a configuration written at umask 022 (mode -rw-r–r–, throwaway keys, endpoint 127.0.0.1:20599 with nothing listening, socks_listen 127.0.0.1:20520): `redsocket-wgtcp run --config config.toml` (PID 82063, stopped after 3 s by `kill 82063`). Result: `WARN redsocket_wgtcp::config] config config.toml is readable by other users (mode 644); it holds the private key; INFO redsocket_wgtcp] our key <8 chars>... to peer <8 chars>...`; connect failures logged without the endpoint; full peer public key 0 matches, private key 0 matches. A second invocation with `preshared-key =` **...** appended exited with `Error: parse config TOML`. The configuration file was deleted afterwards.

B17 Tree integrity. Checksums of all non-vendor files before the re-verification runs and after: identical, so the runs changed nothing. Against the 05b491a snapshot the maintainers' pass changed Cargo.lock, Cargo.toml, README.md, src/bin/server.rs, src/carrier.rs, src/config.rs, src/lib.rs, src/main.rs, src/netstack.rs, src/record.rs, src/server/engine.rs, src/server/peers.rs, src/server/run.rs, src/socks5.rs and src/wg.rs, removed audit-results.txt, and the tree now also holds the centrally rendered PENTEST-REPORT.pdf.

B18 Render check. `pandoc PENTEST-REPORT.md -t html -o /tmp/rvpn01/wp5/render.html` on this version. Result: parses without error.

Appendix B: Dependency inventory and advisories

State at the assessed commit 05b491a: 172 packages in the lockfile, 171 from the crates.io registry with checksums, 0 git sources, 0 path sources other than the crate itself, all vendored under vendor/ and enforced by `.cargo/config.toml` (`replace-with = "vendored-sources"`); cargo audit 0.22.2 against the advisory database of 2026-10-14 (1,329 advisories): 0 vulnerabilities, 1 warning (chacha20 0.10.1 yanked); no rust-toolchain.toml, no `[profile]` section, no SBOM.

State at the remediation HEAD 27a912d (re-verified, B06 and B07): 166 packages, 165 from the registry with checksums, 0 git or path sources, 165 vendored directories. Removed from the graph: rand 0.10.2, rand_core 0.10.1, chacha20 0.10.1 (the yanked release), one of two cpufeatures copies, errno and signal-hook-registry. cargo audit now reports 0 vulnerabilities and 0 warnings. `tun` is optional behind the default `server` feature (Cargo.toml:24-30), so an embedder building with `default-features = false` no longer compiles the TUN crate, nix, wintun-bindings or libloading (verified with `cargo build --offline --locked --no-default-features --lib`, B01); tokio is reduced to `rt-multi-thread`, `net`, `io-util`, `sync`, `time` and `macros` (Cargo.toml:22); `[profile.release] strip = true` is set (Cargo.toml:45-46; the release binary shrank from 16.3 MB with symbols to 3.1 MB). Still open: no rust-toolchain.toml, no `lto`, `codegen-units` or `panic` setting, no SBOM, no CI with cargo audit or cargo deny, and six crates still present in two versions (base64 0.22.1 and 0.23.1, bitflags 1.3.2 and 2.13.1, getrandom 0.2.17 and 0.4.3, syn 2.0.119 and 3.0.3, plus defmt and windows-sys for other targets).

Crate	Version at 27a912d	Role	Advisory
boringtun	0.7.1	Noise IK, transport, timers	none
smoltcp	0.13.1	in-process TCP/IP	none
tokio	1.53.1 (six features)	async runtime	none
x25519-dalek / curve25519-dalek	2.0.1 / 4.1.3	key agreement (via boringtun)	none
chacha20poly1305 / chacha20	0.10.1 / 0.9.1	cookie AEAD (via boringtun)	none
ring	0.17.14	ChaCha20-Poly1305 transport (via boringtun)	none
blake2 / hmac	0.10.6 / 0.12.1	hashing, KDF, v6 derivation	none
getrandom	0.4.3 (direct), 0.2.17 (via rand_core 0.6.4, ring)	OS randomness, session index, jitter	none
zeroize	1.9.0	transitive; unused by the crate	none
toml / serde	1.1.4 / 1.0.229	configuration	none
clap / env_logger / log	4.6.6 / 0.11.11 / 0.4.33	CLI, logging	none
anyhow / thiserror / futures	1.0.104 / 2.0.20 / 0.3.33	errors, server join	none
base64 / hex	0.23.1 (+0.22.1) / 0.4.3	key encoding	none
socket2	0.6.5	protected carrier dial	none
tun (+ nix, wintun-bindings, libloading)	0.8.14, optional	server TUN device only (feature server)	none
rand / chacha20 0.10.1	removed	formerly CLI genkey	yanked crate gone

Occurrences of the token `unsafe` in the vendored sources on the client's path (grep over each crate's src/, including comments; a measure of surface, not of defects): tokio 1,097; nix 925; libc 680; socket2 247; ring 232; wintun-bindings 195; mio 176; curve25519-dalek 117; getrandom 117 (+58 for 0.2.17); tun 96; boringtun

94; parking_lot 88; libloading 68; poly1305 50; blake2 26; chacha20 0.9.1 23; smoltcp 21; x25519-dalek 0; chacha20poly1305 0; redsocket-wgtcp 0. With `default-features = false` the nix, tun, wintun-bindings and libloading entries leave the embedder's build. boringtun's `device`, `ffi-bindings` and `jni-bindings` features are not enabled, so its C ABI and device code are not compiled.

Supply-chain observations against NIST SP 800-218 SSDF v1.1 and SLSA: sources pinned and vendored (PS.1, PW.4 satisfied); no signed provenance or attestation for the published artefacts (SLSA level not claimable from the tree); no SBOM (CycloneDX 1.7 or SPDX 3.0.1 recommended, see RVPN-01-507); no automated vulnerability scanning in the tree (no CI configuration present).

Appendix C: Standards conformance checklist

OWASP ASVS 5.0.0, applied as a library-appropriate subset at chapter and section level. Chapters that presuppose a web application or user-facing authentication are marked Not assessed. The results reflect the assessed commit 05b491a; the remediation status of each referenced finding at HEAD 27a912d is given in that finding's Fix Note.

Control	Result	Note
V1 Encoding and Sanitization	Not assessed	no rendered output; no injection surface
V2.2 Input Validation	Partial	config and record parsers robust (A9, A11); endpoint, unknown keys, MTU gaps (RVPN-01-514); unvalidated in-memory path (RVPN-01-509)
V2.3 Business Logic Security	Partial	readiness flag precedes handshake (RVPN-01-502); SOCKS reply precedes connection (RVPN-01-513)
V2.4 Anti-automation	Fail	no stream cap or request timeout on the SOCKS5 front (RVPN-01-504); server DH before mac1, no cookies (RVPN-01-512)
V3 Web Frontend Security	Not assessed	not applicable
V4 API and Web Service	Not assessed	not applicable
V5 File Handling	Not assessed	only the configuration file; see V13
V6 Authentication	Partial	peer authentication by Noise IK with pinned static key: pass; local SOCKS5 front unauthenticated (RVPN-01-504)
V7 Session Management	Not assessed	no user sessions; WireGuard sessions under V11
V8 Authorization	Partial	server anti-spoofing and authenticated roaming: pass; any local process may use the proxy (RVPN-01-504)
V9 Self-contained Tokens	Not assessed	not applicable
V10 OAuth and OIDC	Not assessed	not applicable
V11.2 Secure Cryptography Implementation	Partial	X25519, ChaCha20-Poly1305, BLAKE2s via boringtun; protocol deviations in padding, jitter, message limits (RVPN-01-515)
V11.3 Encryption Algorithms	Pass	AEAD with per-session keys and counters; replay window verified (A10, A15)

Control	Result	Note
V11.4 Hashing and Hash-based Functions	Pass	BLAKE2s HMAC KDF and keyed MACs as specified; domain-separated address derivation
V11.5 Random Values	Pass	getrandom for keys; OS-seeded ThreadRng for the CLI; deterministic session index is a protocol, not an RNG, defect (RVPN-01-501)
V11.6 Public Key Cryptography	Pass	static key pinning; PSK supported; TAI64N replay check verified
V12 Secure Communication	Fail	carrier identifiable from the first segment (RVPN-01-501); names resolved outside the tunnel (RVPN-01-503); payload itself encrypted and authenticated
V13.2 Backend Communication Configuration	Partial	endpoint validation weak (RVPN-01-514); no non-loopback bind guard (RVPN-01-504)
V13.3 Secret Management	Fail	private key in Debug and Serialize output, not zeroised, file mode unchecked (RVPN-01-506)
V13.4 Unintended Information Leakage	Partial	build paths remapped: pass; test values outside documentation ranges (RVPN-01-510)
V14.2 General Data Protection	Partial	no files written; keys and destinations reach the log (RVPN-01-505)
V15.2 Security Architecture and Dependencies	Partial	vendored, checksummed, 0 advisories; yanked crate, duplicates, no pin, no SBOM (RVPN-01-507)
V15.3 Defensive Coding	Partial	no unsafe; panics reachable from embedder input (RVPN-01-509); parser quadratic (RVPN-01-508)
V15.4 Safe Concurrency	Fail	single event loop blocks on the socket write (RVPN-01-511)
V16.2 General Logging	Partial	private key never logged; public keys, endpoint and destinations logged (RVPN-01-505)
V16.5 Error Handling	Partial	fail-closed carrier with bounded back-off: pass; readiness misreported (RVPN-01-502); watchdog silenced by the stall (RVPN-01-511)
V17 WebRTC	Not assessed	not applicable

Appendix D: Open questions for the maintainers

1. What does a record with `WG_TCP_FRAG_FLAG` carry on the kernel side? The client strips the 4-byte fragment header and passes the remainder to boringtun as a complete WireGuard message (record.rs:167-168). If the kernel module ever splits one WireGuard message across fragment records, the client silently drops it after an AEAD failure. Verification needs the kernel source (WP6).

2. Does the kernel-module server behave like the userspace one under probing (accept, stay silent, never close), and does it emit the same first bytes `00 00 00 64 00 00 a9 25 02 00 00 00`? Verification needs a kernel host (WP6).
3. Which readiness signal do the Android and iOS embedders consume: `carrier_is_up()` only, or also `time_since_last_handshake()`? (WP3, WP4; decides the effective severity of RVPN-01-502.)
4. Do any embedders send ATYP 0x03 host names to the SOCKS5 front, or only IP literals? (Decides the effective severity of RVPN-01-503.)
5. Is `wg_tcp_encap_header` frozen, or can a keyed, padded framing or a cover protocol be rolled out to both ends together, and which wrapper is planned?
6. Is redsocket-wgtcp-server deployed anywhere reachable from the Internet? (Decides whether RVPN-01-512 and RVPN-01-516 should be raised to Medium.)
7. What is the provenance of the key pair at src/wg.rs:187-188? If it was ever used outside the test, it should be rotated.
8. Is there a reason `rand` must remain when `generate_keypair()` already uses `getrandom`?
9. Is it intended that streams survive a carrier loss untouched and that their smoltcp timers do not run while the carrier is down (carrier.rs:112, :190)? If so, document it; if not, abort or time out the streams on carrier loss.
10. What is the observed time to detect a fully black-holed carrier on Android and iOS (host TCP retransmission limit), given that the WireGuard watchdog does not run while a write is blocked (RVPN-01-511)?
11. Is a client that never sends data (keepalives only) expected to re-handshake at 180 s, when boringtun drops the session at REJECT_AFTER_TIME, or only at 540 s (REJECT_AFTER_TIME x 3, timers.rs:208-213)? Every run in this assessment sent data, so the 120 s rekey pre-empted both; a pure-idle run was not performed.